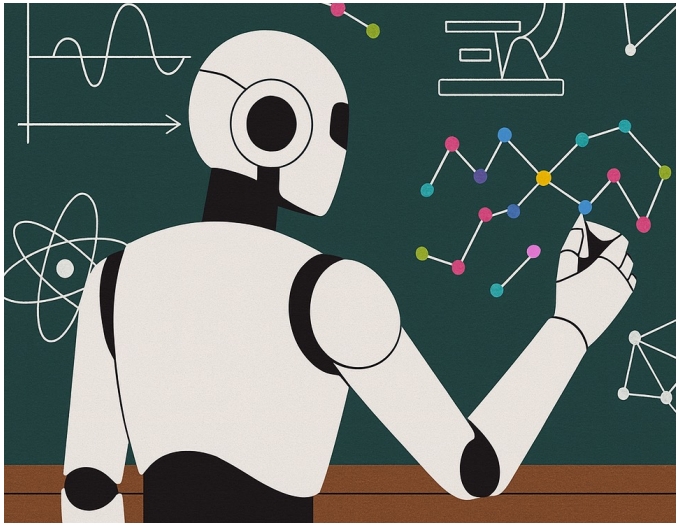
Winter semester 2025/26

AI for Quantum Physics



Xuemei Gu
xuemei.gu@uni-jena.de

If you have any questions, feel free to reach out!

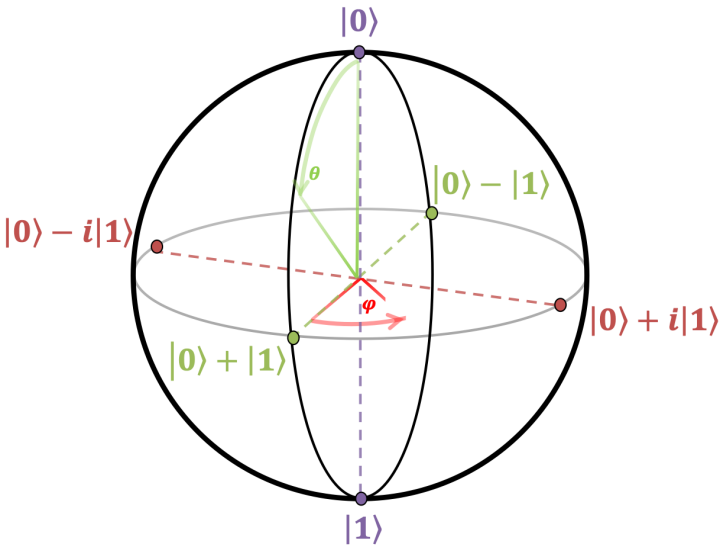
Plan for today:

- Gradient descent Optimization
- Variants of Gradient Descent
- Experiment Design & Optimization
- Discussion

Recap of Last Lecture

Qubit: $|\Psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$ $|0\rangle \equiv \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, $|1\rangle \equiv \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ $\langle 0| \equiv (1 \;\; 0)$, $\langle 1| \equiv (0 \;\; 1)$.

$|\alpha|^2 + |\beta|^2 = 1$



Bloch sphere of two-dimensional degrees of freedom

Projective measurement is defined as an operation that projects a quantum state onto a specific outcome

projective measurement

$$\langle 0|\psi\rangle = \langle 0|\left(\alpha|0\rangle + \beta|1\rangle\right)$$
$$= \alpha\langle 0|0\rangle + \beta\langle 0|1\rangle$$

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \xrightarrow[|0\rangle/|1\rangle]{P=|\alpha|^2} |0\rangle$$
$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \xrightarrow[|0\rangle/|1\rangle]{P=|\beta|^2} |1\rangle$$

Recap of Last Lecture

Two qubit system case

Separable State:

$$|\Psi\rangle_{AB} = |\psi\rangle_A \otimes |\phi\rangle_B$$

Entangled State:

$$|\Psi\rangle_{\text{ent}} \neq |\psi\rangle_A \otimes |\phi\rangle_B$$

General two qubit state: $|\psi\rangle = a|00\rangle + b|01\rangle + c|10\rangle + d|11\rangle$

$a=b=\frac{1}{\sqrt{2}} \quad c=d=0$

$$\Rightarrow |\psi\rangle_{AB} = \frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|01\rangle = \frac{1}{\sqrt{2}}|0\rangle \otimes (|0\rangle + |1\rangle)$$

$$a=d=\frac{1}{\sqrt{2}},\ b=c=0 \Rightarrow |\psi\rangle_{AB} = \frac{|00\rangle + |11\rangle}{\sqrt{2}} \neq A \otimes B$$

Recap of Last Lecture

Schmidt-Rank Vector: Classification of high-dimensional multipartite entanglement

new types of entanglement, that are only possible if both n>2 and d>2.

$$\text{SRV } d_\psi = (r_A, r_B, r_C)$$

$$\begin{aligned} r_A &= \text{rank}(\text{Tr}_A(|\psi\rangle\langle\psi|)) \\ r_B &= \text{rank}(\text{Tr}_B(|\psi\rangle\langle\psi|)) \\ r_C &= \text{rank}(\text{Tr}_C(|\psi\rangle\langle\psi|)) \end{aligned}$$

Example states for SRV(3,3,2)

$$|\Psi_{332}\rangle = \frac{1}{\sqrt{3}}(|000\rangle + |111\rangle + |221\rangle)$$

Example states for SRV(3,3,3)

$$|\Psi_{333}\rangle = \frac{1}{\sqrt{3}}(|000\rangle + |111\rangle + |222\rangle)$$

Recap of Last Lecture

Example states for SRV(3,3,2)

$$|\psi\rangle = \frac{1}{\sqrt{3}}(|000\rangle + |111\rangle + |221\rangle)$$

$$\begin{aligned} r_A &= \text{rank}(\text{Tr}_A(|\psi\rangle\langle\psi|)) \\ r_B &= \text{rank}(\text{Tr}_B(|\psi\rangle\langle\psi|)) \\ r_C &= \text{rank}(\text{Tr}_C(|\psi\rangle\langle\psi|)) \end{aligned}$$

density matrix is **an alternate representation of the state of a quantum system**

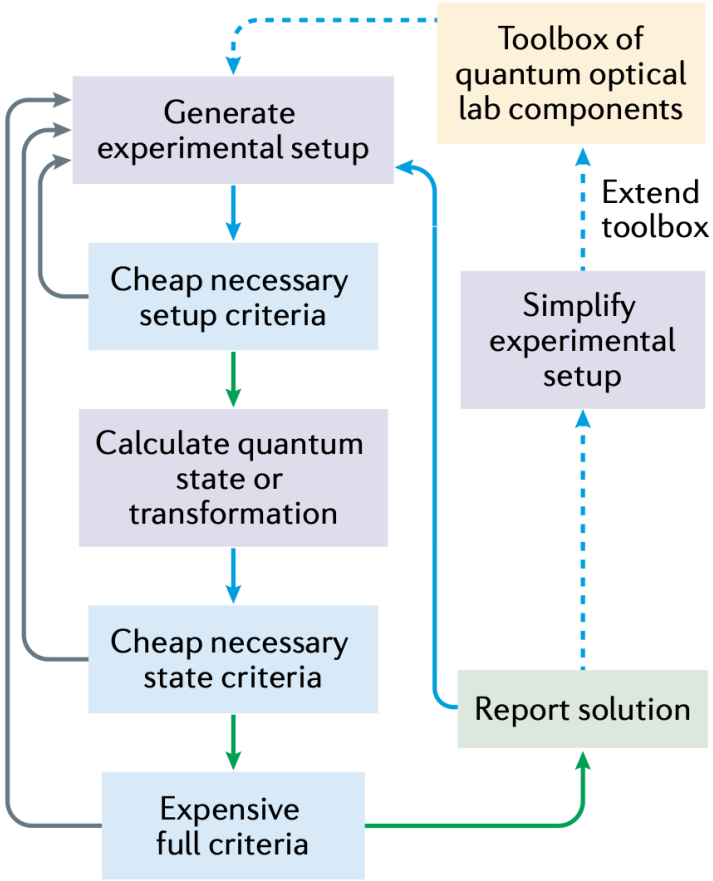
$$\begin{aligned} \rho &= |\psi\rangle\langle\psi| \\ &= \frac{1}{\sqrt{3}}(|000\rangle + |111\rangle + |221\rangle) \\ &\quad \times \frac{1}{\sqrt{3}}(\langle 000| + \langle 111| + \langle 221|) \\ &= \frac{1}{3}\Big[|000\rangle\langle 000| + |000\rangle\langle 111| + |000\rangle\langle 221| \\ &\quad + |111\rangle\langle 000| + |111\rangle\langle 111| + |111\rangle\langle 221| \\ &\quad + |221\rangle\langle 000| + |221\rangle\langle 111| + |221\rangle\langle 221|\Big] \end{aligned}$$

$$Tr_A(\rho) = Tr_{BC}(\rho) = \frac{1}{3}\Big(|0\rangle\langle 0| + |1\rangle\langle 1| + |2\rangle\langle 2|\Big)$$

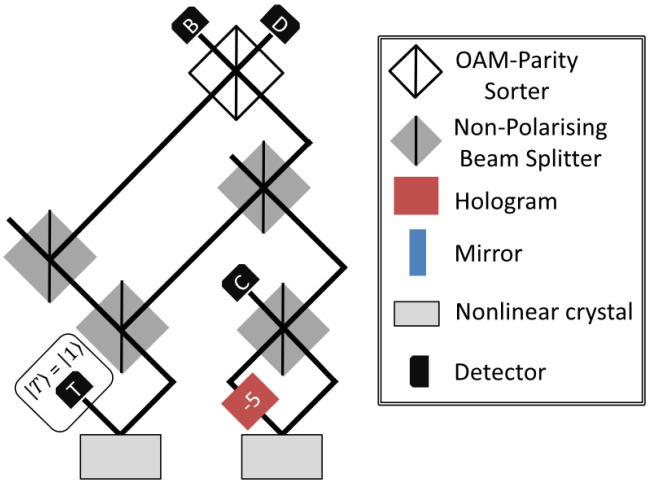
$$\begin{aligned} &\text{rank}(Tr_A(\rho)) \\ &= \text{rank}(Tr_{BC}(\rho)) = 3 \end{aligned} \qquad r_A = 3$$

$$\begin{pmatrix} \frac{1}{3} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{3} \end{pmatrix}$$

Recap of Last Lecture



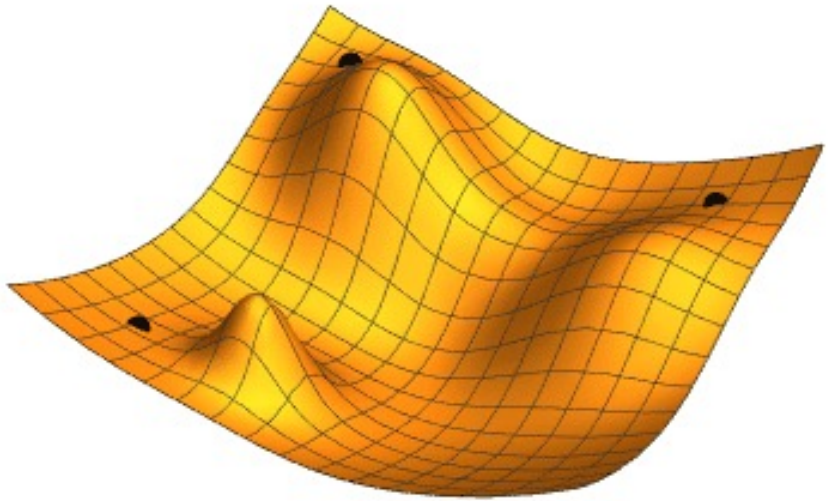
Topological search/Random search



INFINITE AND DISCRETE EXPLORATION SPACE!

Can we do better?

Gradient Descent Optimization



What is optimization?

Optimization means finding the “best” solution to a problem under certain conditions. It involves adjusting variables to minimize costs or maximize gains, efficiency, or performance.



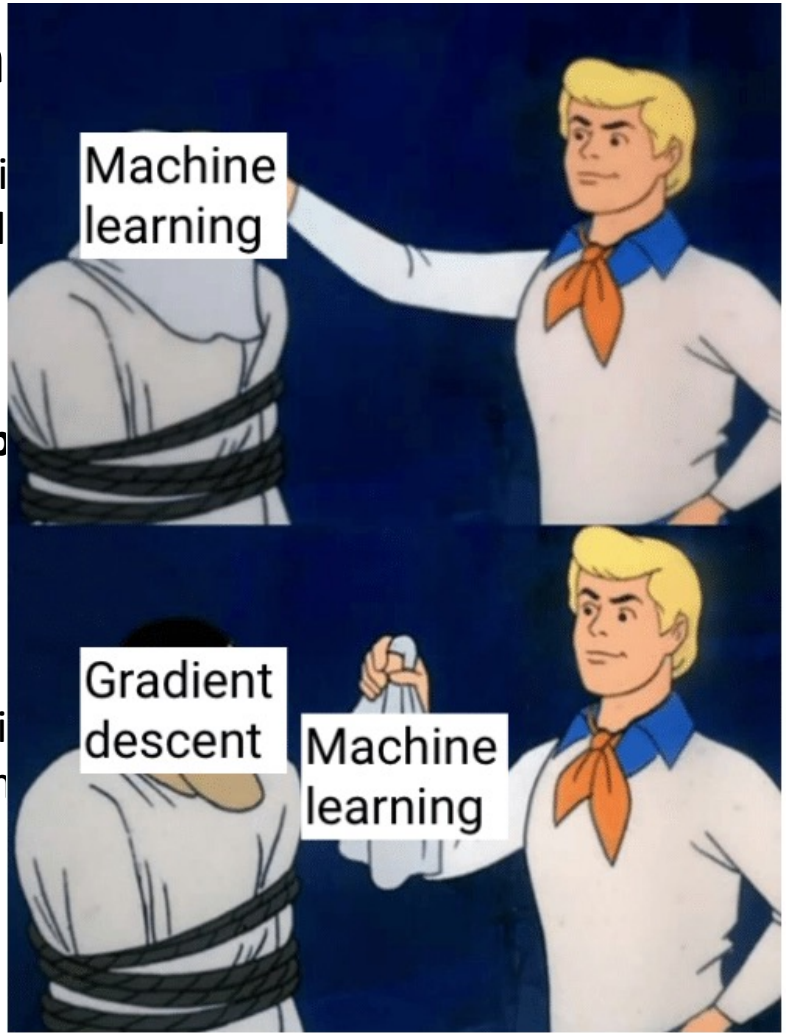
Many learning algorithm in AI, from linear regression to deep quantum networks, is secretly solving a minimization problem. It’s all about finding the bottom of a loss landscape.

Gradient descent

What is optimiza

Optimization means findi er certain conditions. It involves adjusting variabl efficiency, or performance.

Inp t



Many learning algorithm i ntum networks, is secretly solving a minimization m of a loss landscape.

What is Gradient descent?

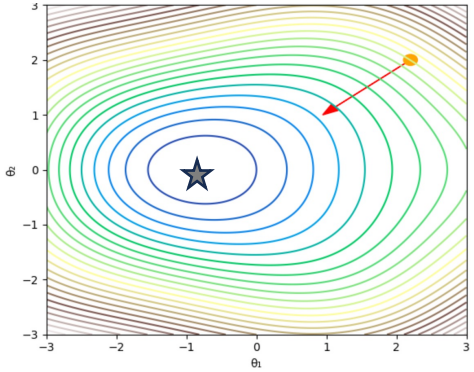
It is a first-order iterative optimization algorithm for finding the minimum of a function



The Scoreboard (Cost Function)

The Compass (The Gradient)

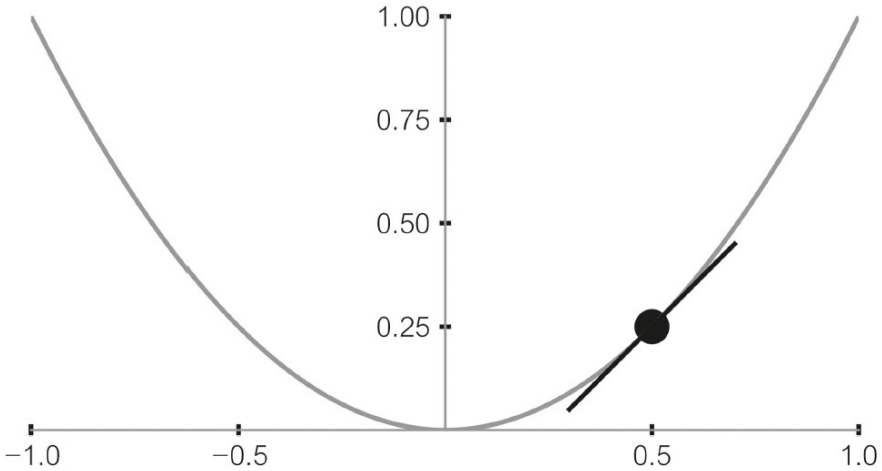
The Step Size (Learning Rate)



Gradient descent

Let’s look at going down a slope for a 2D curve, given by the equation

$$y = f(x) = x^2$$



Gradient descent

$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

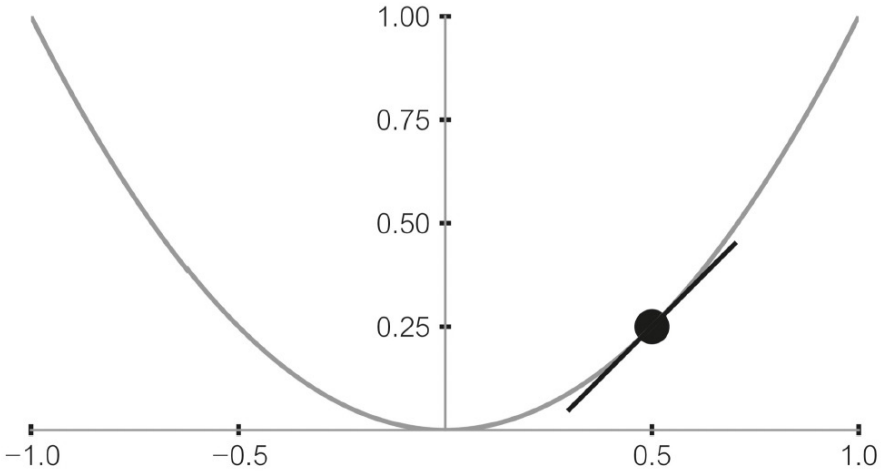
Let's look at going down a slope for a 2D curve, given by the equation

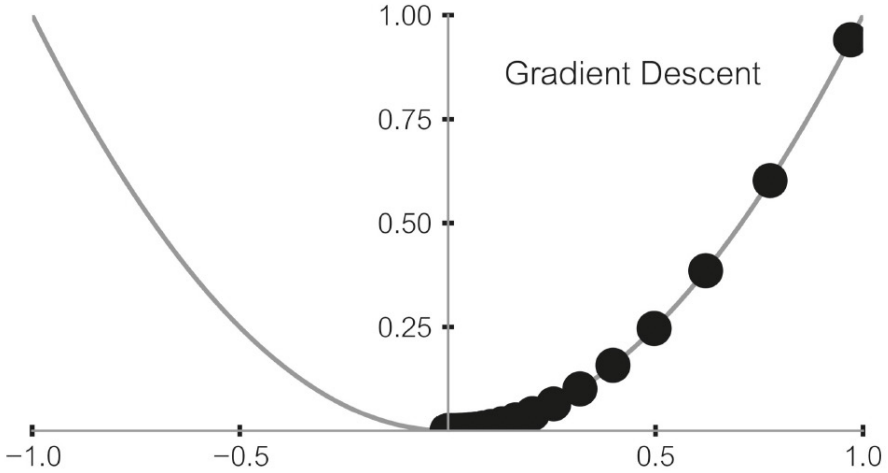
$$y = f(x) = x^2$$

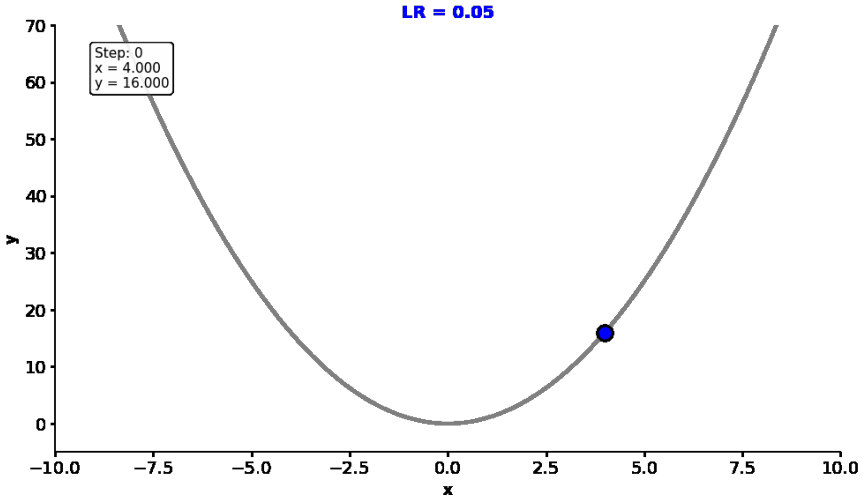
$$\nabla f(x) = \frac{dy}{dx} = 2x$$

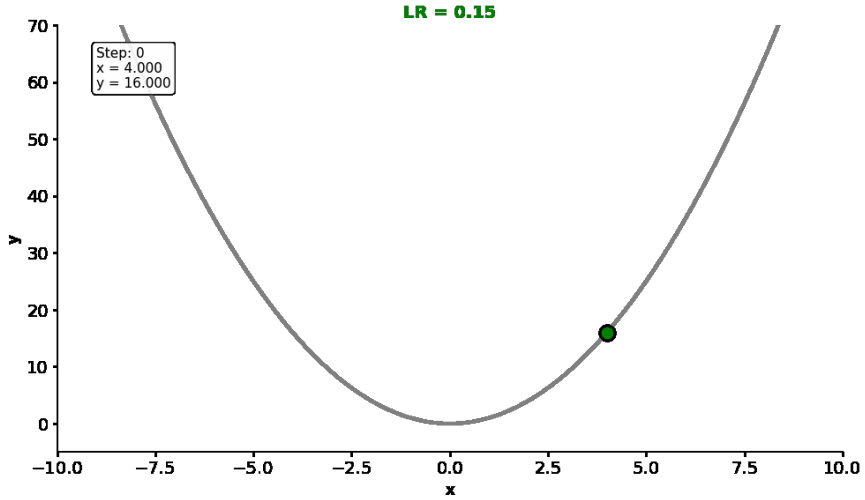
$$x_{\text{new}} = x_{\text{old}} - \eta \nabla_x f(x_{\text{old}})$$

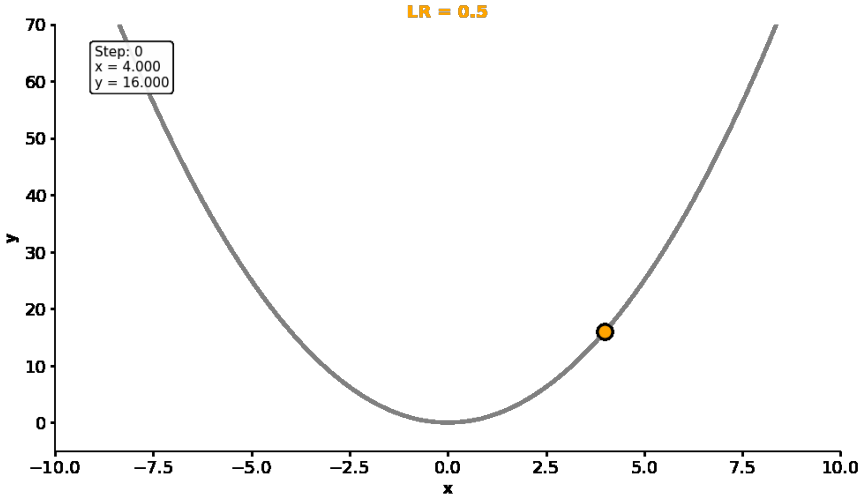
$$y_{\text{new}} = x_{\text{new}}^2$$

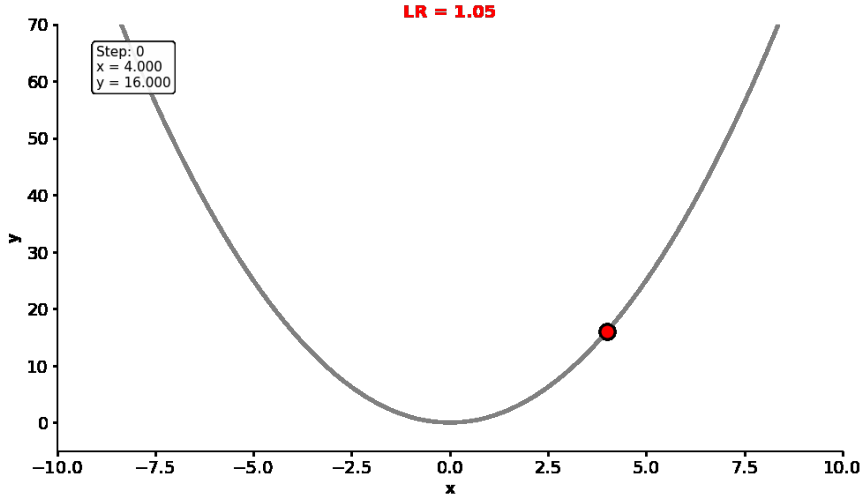





LR = 0.05
Step: 0
x = 4.000
y = 16.000
70
60
50
40
30
20
10
0
y
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
x


LR = 0.15
Step: 0
x = 4.000
y = 16.000
70
60
50
40
30
20
10
0
y
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
x


LR = 0.5
Step: 0
x = 4.000
y = 16.000
70
60
50
40
30
20
10
0
y
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
x


LR = 1.05
Step: 0
x = 4.000
y = 16.000
70
60
50
40
30
20
10
0
y
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
x

Accelerating Gradient Descent

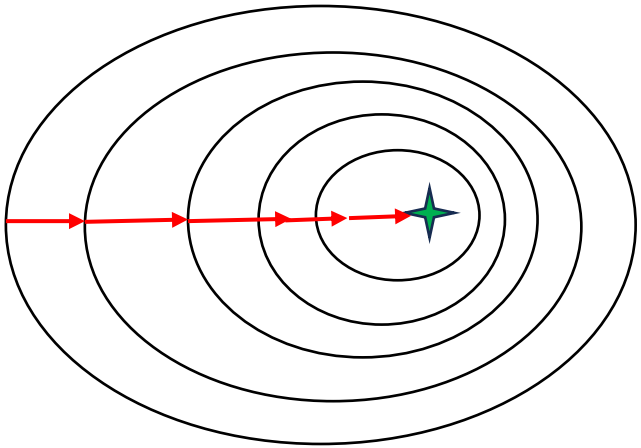
Normal Gradient Descent (pure optimization or in ML case)

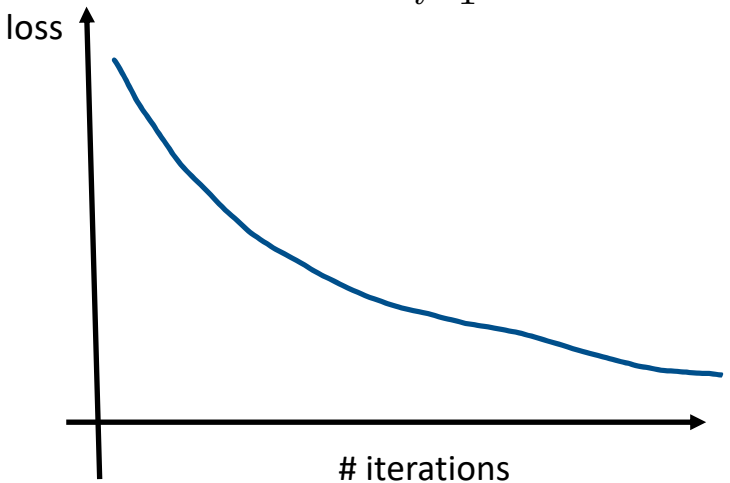
$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

x_t : current value of the variable
η : learning rate (step size)
$\nabla f(x_t)$: gradient of f at x_t

We need to calculate the gradient for every single one of those millions of sub-functions and add them all up. This is incredibly slow.

$$f(x) = \frac{1}{N} \sum_{i=1}^{N} f_i(x)$$



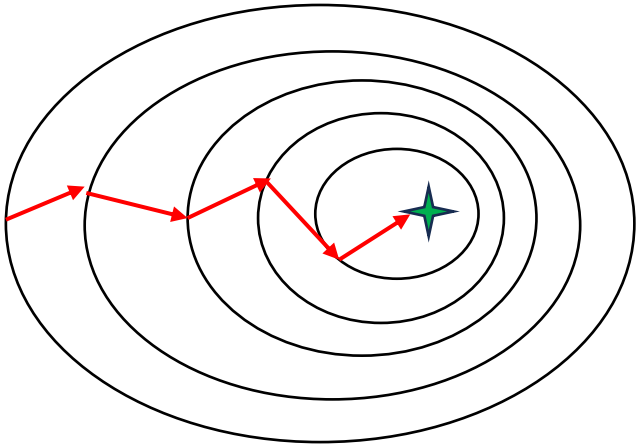


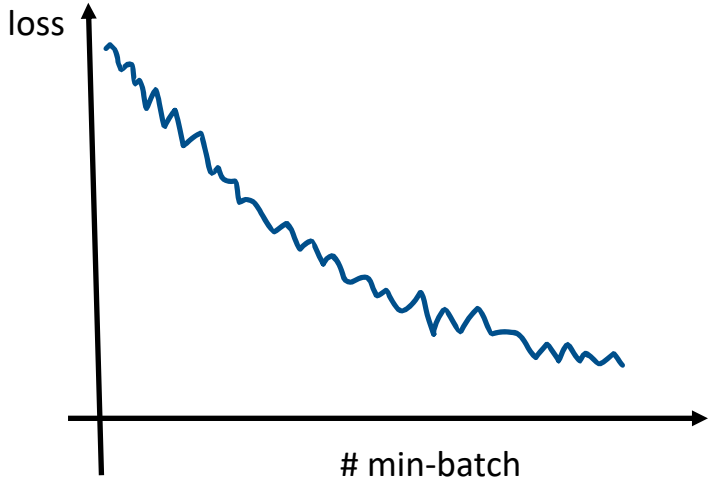
Accelerating Gradient Descent

Min-Batch Gradient Descent (ML case)

$$x_{t+1} = x_t - \eta \nabla_B f(x_t)$$

$$f(x)_B = \frac{1}{B} \sum_{i \in B} f_i(x)$$



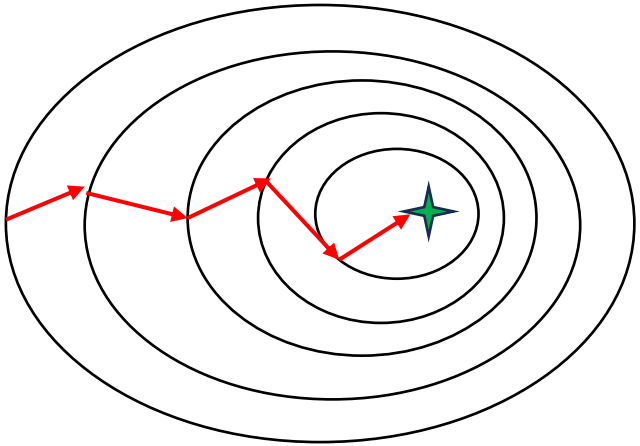


Accelerating Gradient Descent

Min-Batch Gradient Descent (ML case)

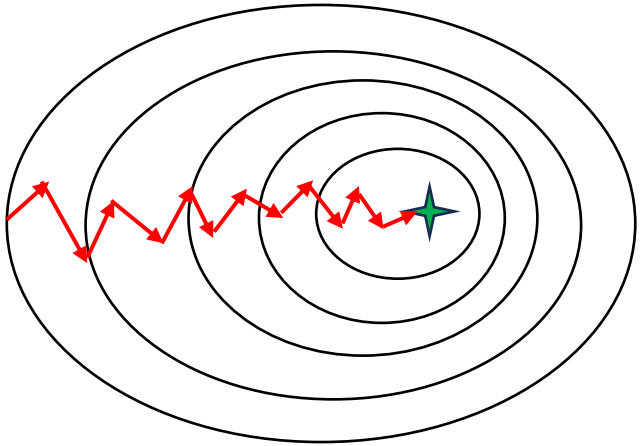
$$x_{t+1} = x_t - \eta \nabla_B f(x_t)$$

$$f(x)_B = \frac{1}{B} \sum_{i \in B} f_i(x)$$



Stochastic Gradient Descent

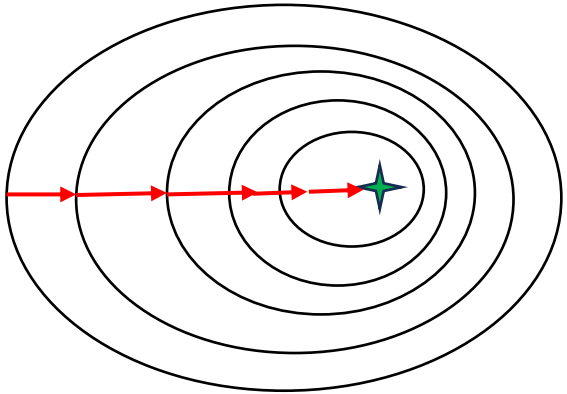
$$x_{t+1} = x_t - \eta \nabla f_i(x_t)$$



Accelerating Gradient Descent in ML case

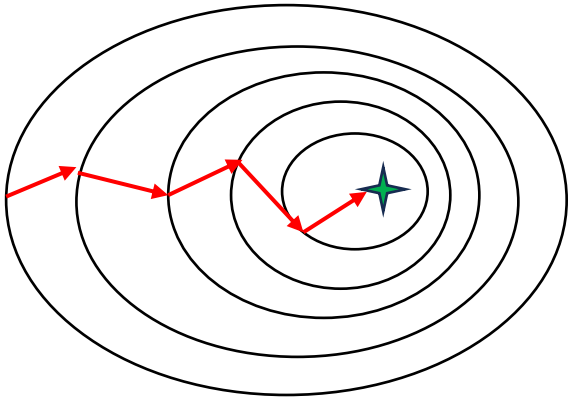
Normal Gradient Descent

$$x_{t+1} = x_t - \eta \nabla f(x_t)$$



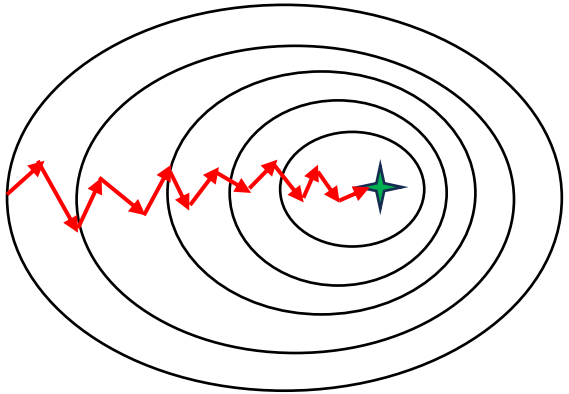
Min-Batch Gradient Descent

$$x_{t+1} = x_t - \eta \nabla_B f(x_t)$$

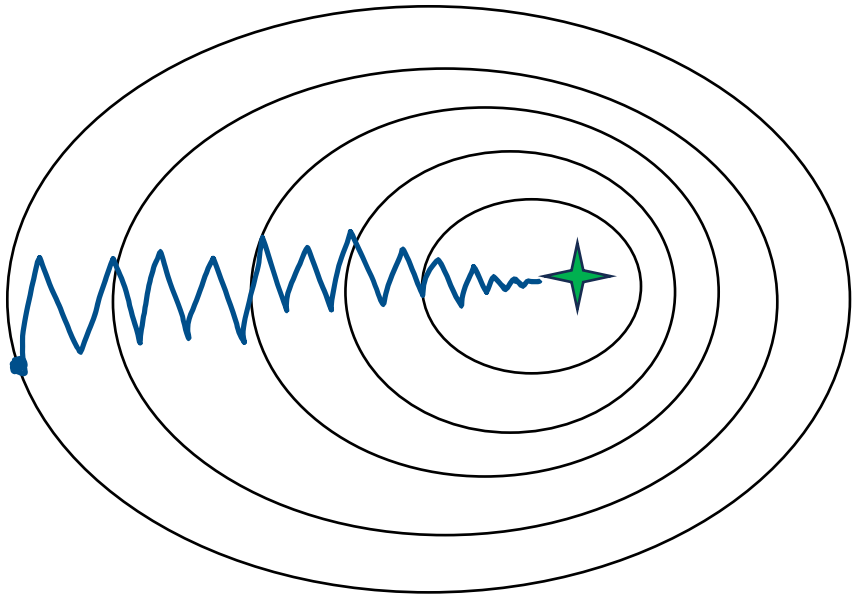


Stochastic Gradient Descent

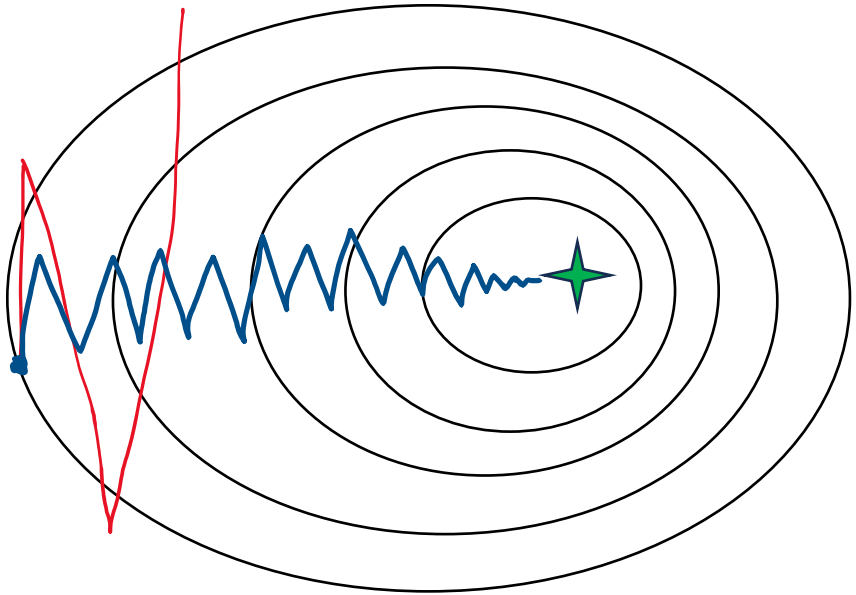
$$x_{t+1} = x_t - \eta \nabla f_i(x_t)$$



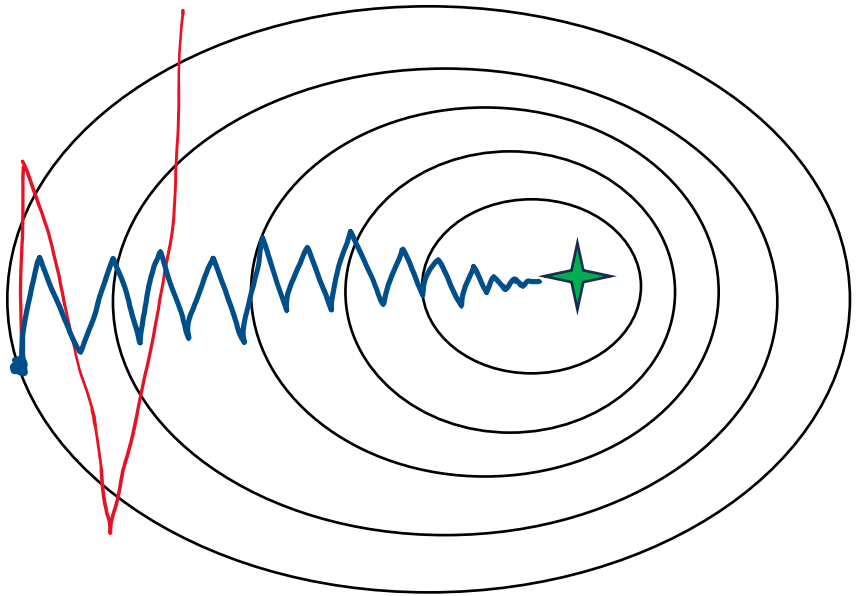
$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

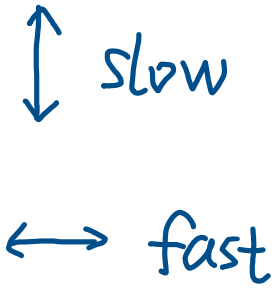


$$x_{t+1} = x_t - \eta \nabla f(x_t)$$



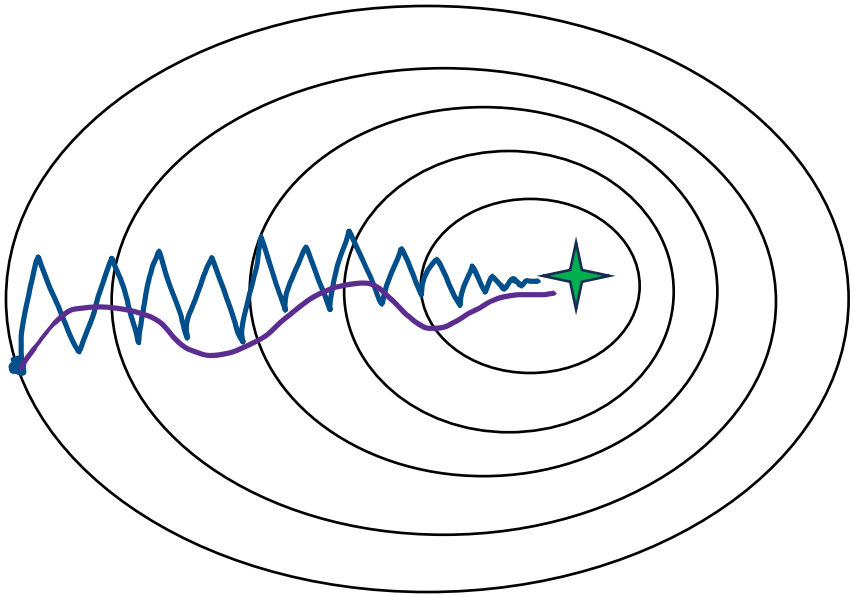
$$x_{t+1} = x_t - \eta \nabla f(x_t)$$





Gradient Descent with Momentum

$$x_{t+1} = x_t - \eta \nabla f(x_t)$$



carries **memory** of previous gradients

adds the **new gradient** information

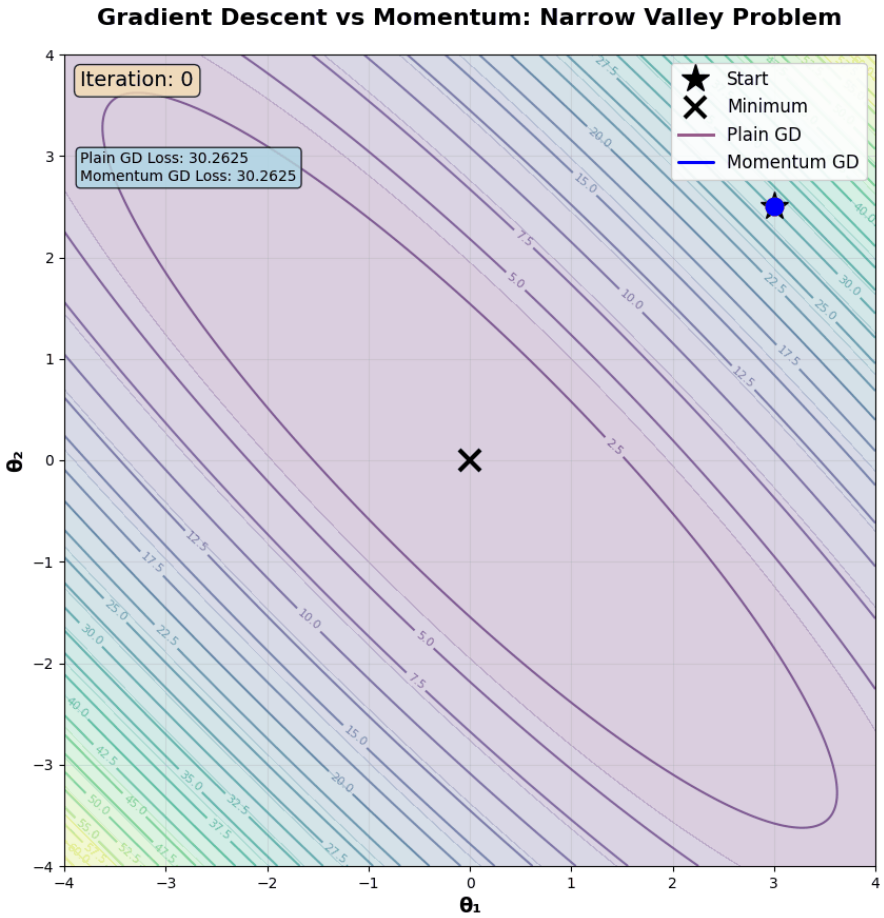
$$v_{t+1} = \beta v_t + (1 - \beta) \nabla f(w_t)$$

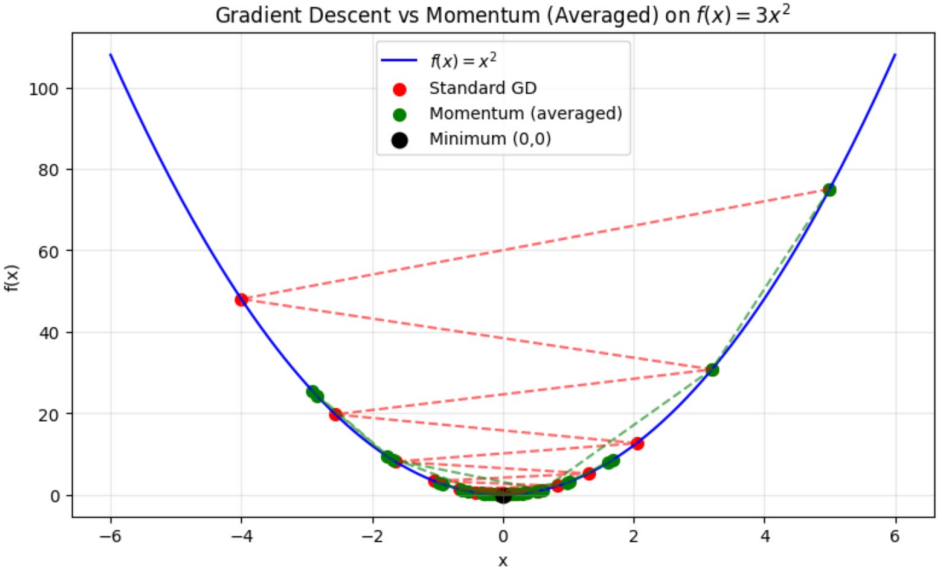
$$w_{t+1} = w_t - \eta\, v_{t+1}$$

w_t are the parameters at iteration t,
$\nabla f(w_t)$ is the gradient of the loss function with respect to w_t,
v_t is the exponentially smoothed gradient (the "momentum"),
η is the learning rate,
$\beta \in [0, 1)$ is the momentum coefficient (e.g. $\beta = 0.9$).

Momentum is a concept from physics where an object's motion depends not only on the current force but also on its previous velocity. In the context of gradient optimization, it refers to a method that smoothens the optimization trajectory by adding a term that helps the optimizer remember the past gradients.

Gradient Descent with Momentum





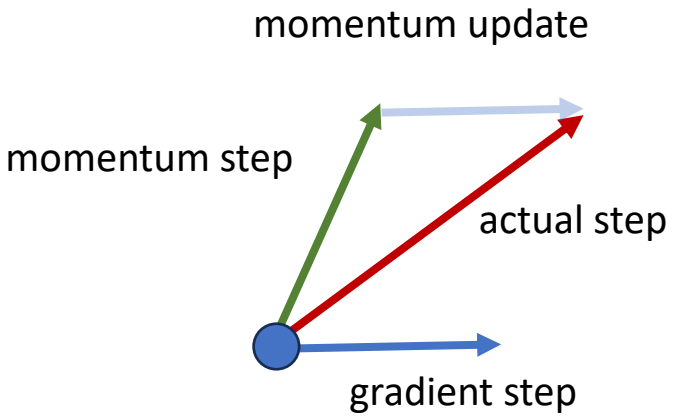
Nesterov Accelerated Gradient

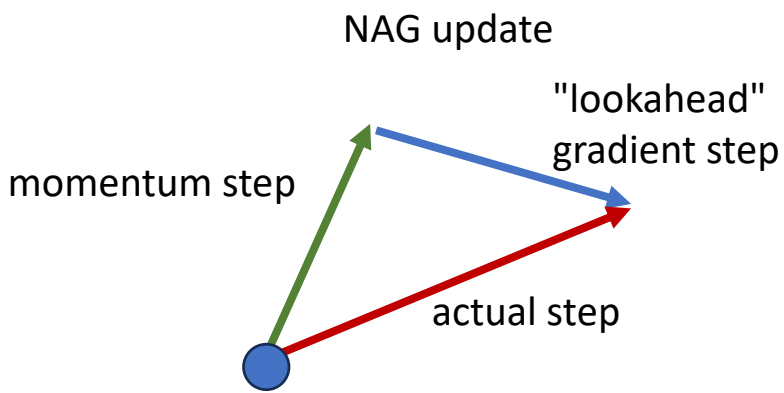
Momentum methods can "overshoot." Nesterov's insight is to take a look-ahead step before measuring the gradient, so one can correct earlier and aim where you are *going*, not where you *are*

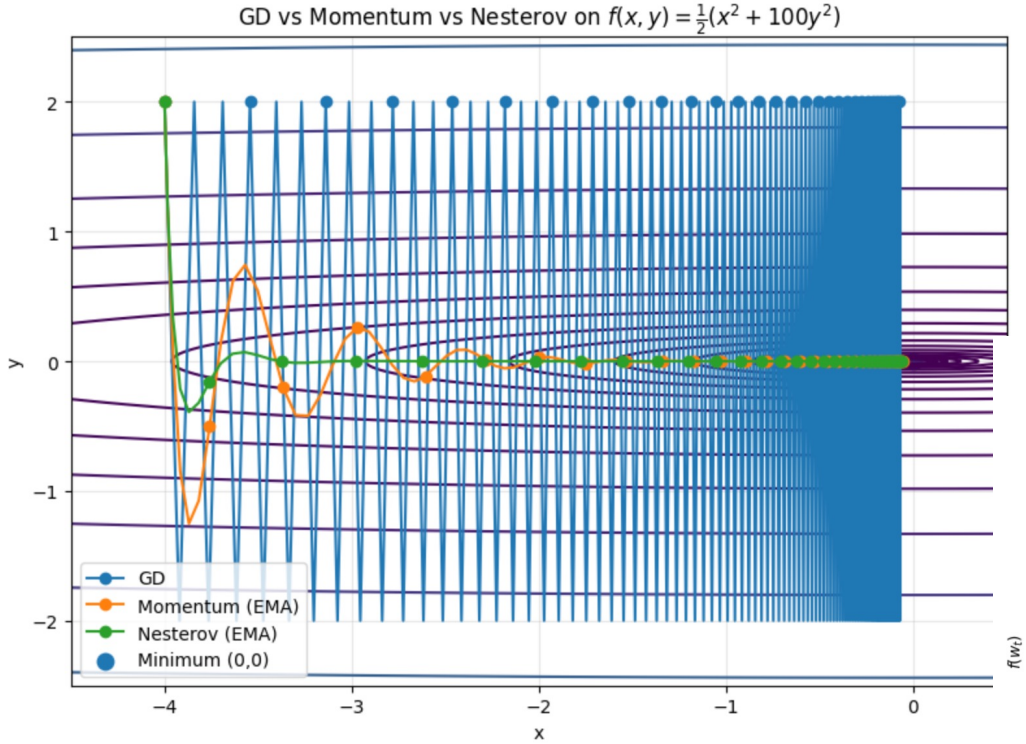
"lookahead" gradient step $\tilde{w}_t = w_t - \eta \beta v_t$

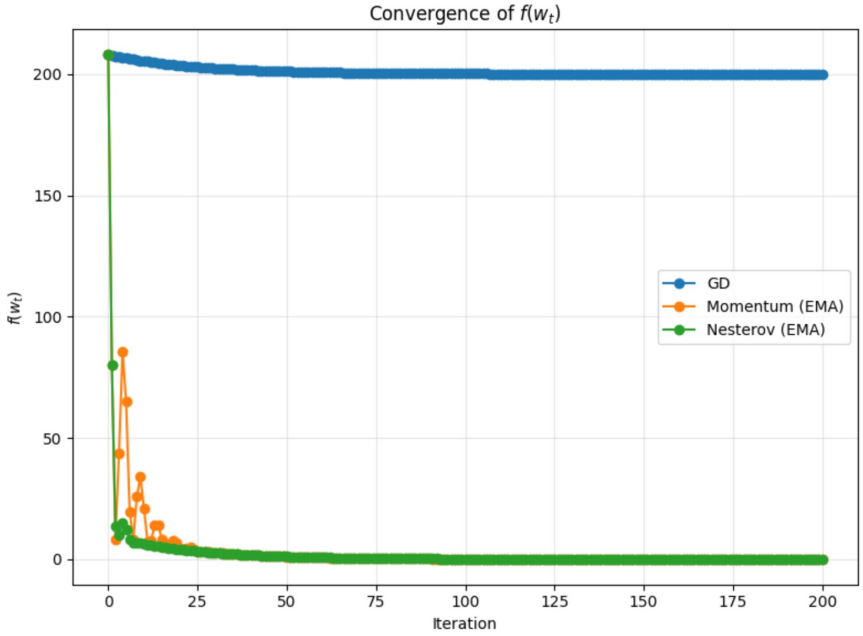
$$v_{t+1} = \beta v_t + (1 - \beta) \nabla f(\tilde{w}_t)$$

$$w_{t+1} = w_t - \eta v_{t+1}$$






GD vs Momentum vs Nesterov on $f(x, y) = \frac{1}{2}(x^2 + 100y^2)$
GD
Momentum (EMA)
Nesterov (EMA)
Minimum (0,0)
x
y


Convergence of $f(w_t)$
GD
Momentum (EMA)
Nesterov (EMA)
Iteration
$f(w_t)$

AdaGrad (Adaptive Gradient)

Adjust each parameter's learning rate automatically based on its gradient history.

$$g_t = \nabla f(w_t)$$

$$r_{t+1} = r_t + g_t^2$$

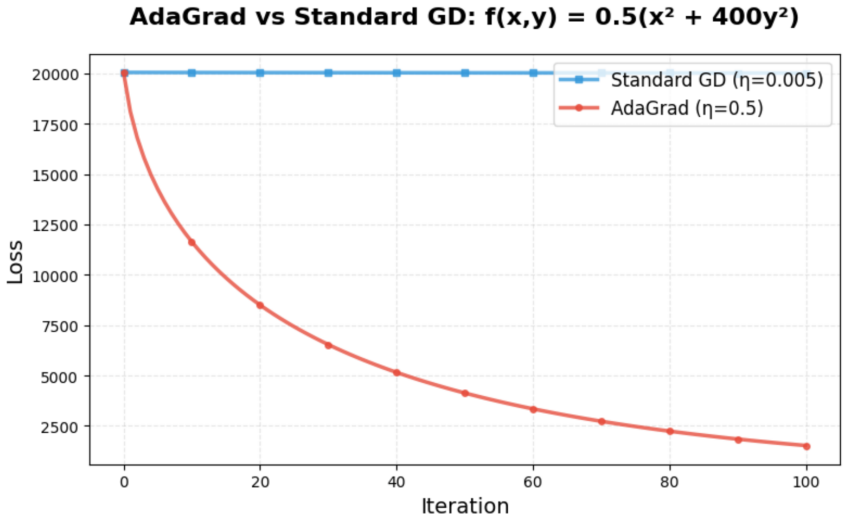
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{r_{t+1}} + \epsilon} g_t$$

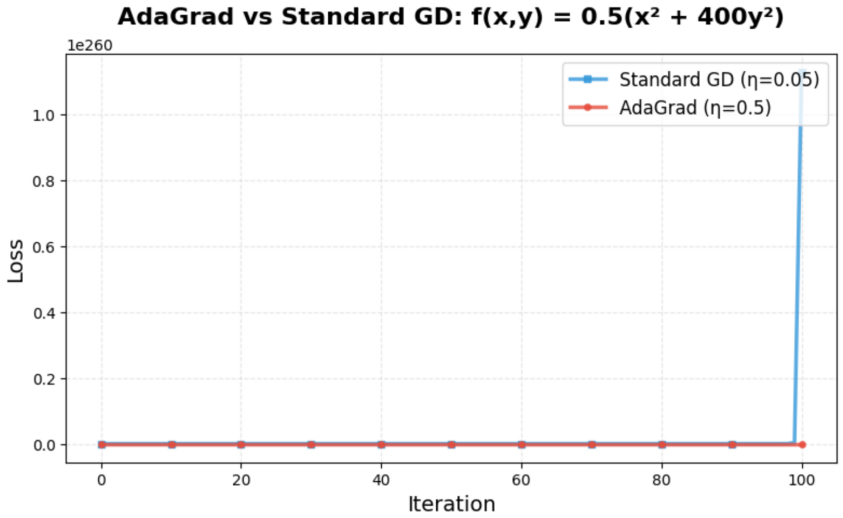
g_t: current gradient

r_t: sum of past squared gradients

η: base learning rate

ϵ: small constant for stability





RMSProp (Root Mean Square Propagation)

Fixes AdaGrad's aggressive learning rate decay by adding exponential decay

$$g_t = \nabla f(w_t)$$

$$v_t = \beta\, v_{t-1} + (1-\beta)\, g_t^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon}\, g_t$$

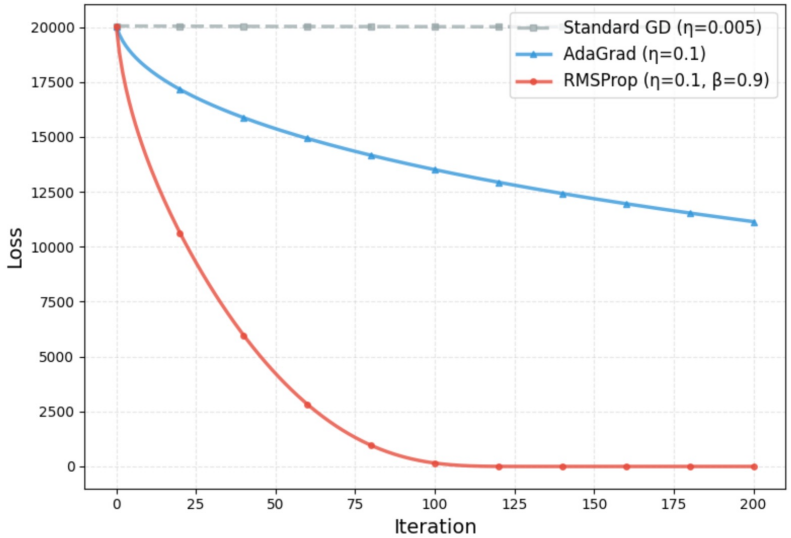
v_t: exponential moving average of squared gradients

$\rho \in [0, 1)$: decay factor (typical $\rho \approx 0.9$)

η: base learning rate, ϵ: small constant for stability

AdaGrad: $r_t = r_{t-1} + g_t^2$ (monotone growth) → step sizes can vanish.

RMSProp: $v_t = \beta v_{t-1} + (1-\beta) g_t^2$ (decay) → step sizes stabilize.



Adam (Adaptive Moment Estimation)

First moment (mean of gradients)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t$$

Second moment (mean of squared gradients)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Update rule

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

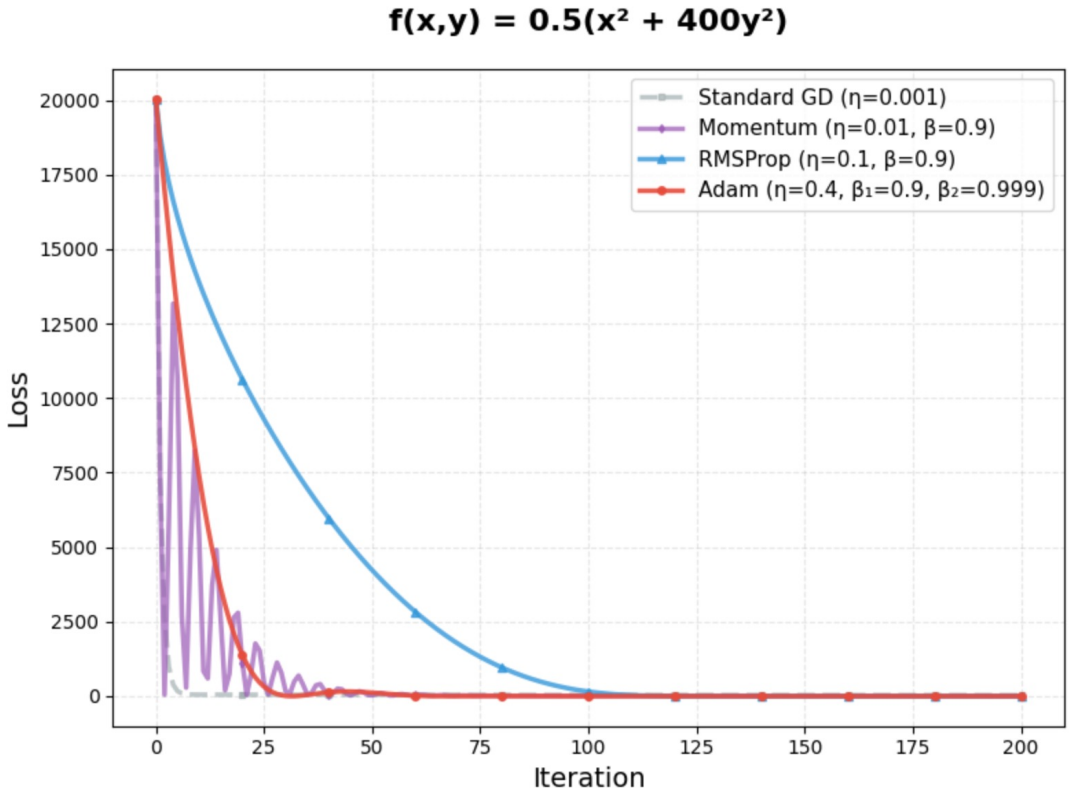
Adam (Adaptive Moment Estimation)

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t$$

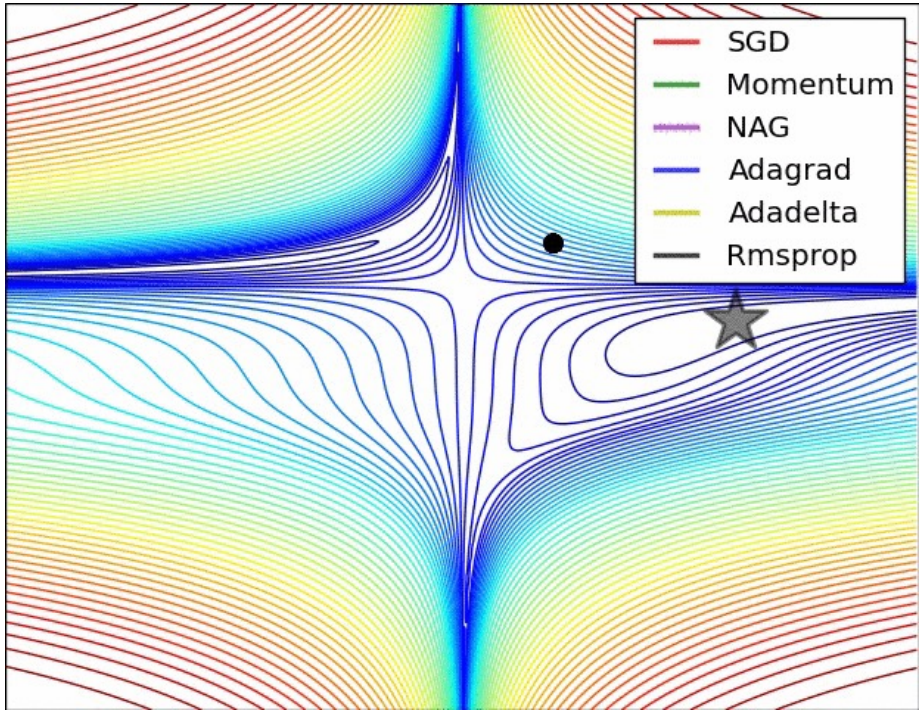
$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2$$

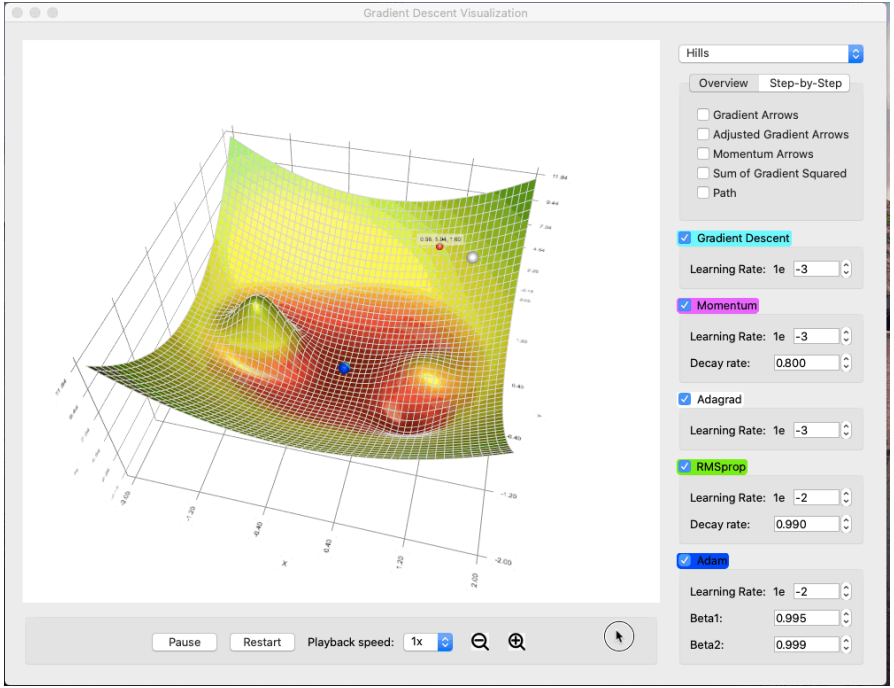
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$



https://github.com/lilipads/gradient_descent_viz?tab=readme-ov-file





https://arxiv.org/abs/1609.04747

Second-order optimization

Second-order optimization methods use not only the **gradient** (first derivative) but also the **Hessian matrix** — the matrix of *second derivatives* — to capture **curvature** of the loss function

For a function $f(\boldsymbol{x}) : \mathbb{R}^n \to \mathbb{R}$, the Hessian is defined as: $\boldsymbol{H}_{ij} = \frac{\partial^2 f}{\partial x_i \, \partial x_j}$

$$\boldsymbol{H}(\boldsymbol{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

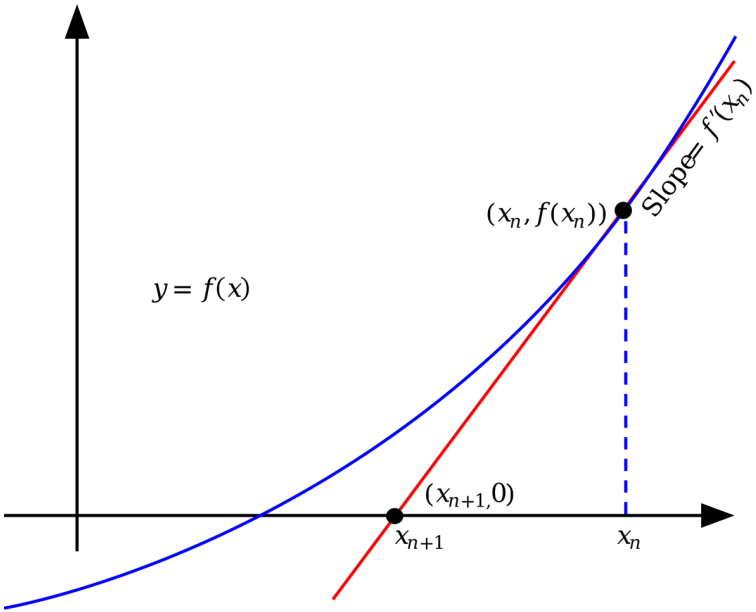
Second-order optimization

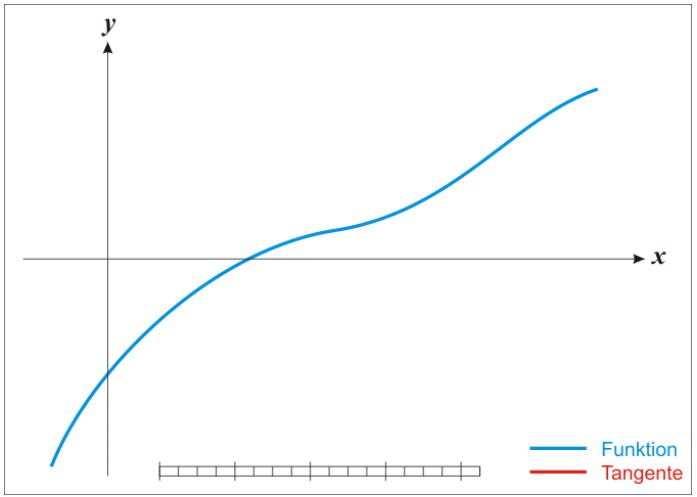
Newton's Method (Newton–Raphson method)

The idea is to start with an initial guess near a root, approximate the function by its tangent line near the guess, and then take the root of the linear approximation as a next guess at the function's root. This will typically be closer to the function's root than the previous guess, and the method can be iterated

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - H(\boldsymbol{x}_k)^{-1} \nabla f(\boldsymbol{x}_k)$$

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - f' / f''$$



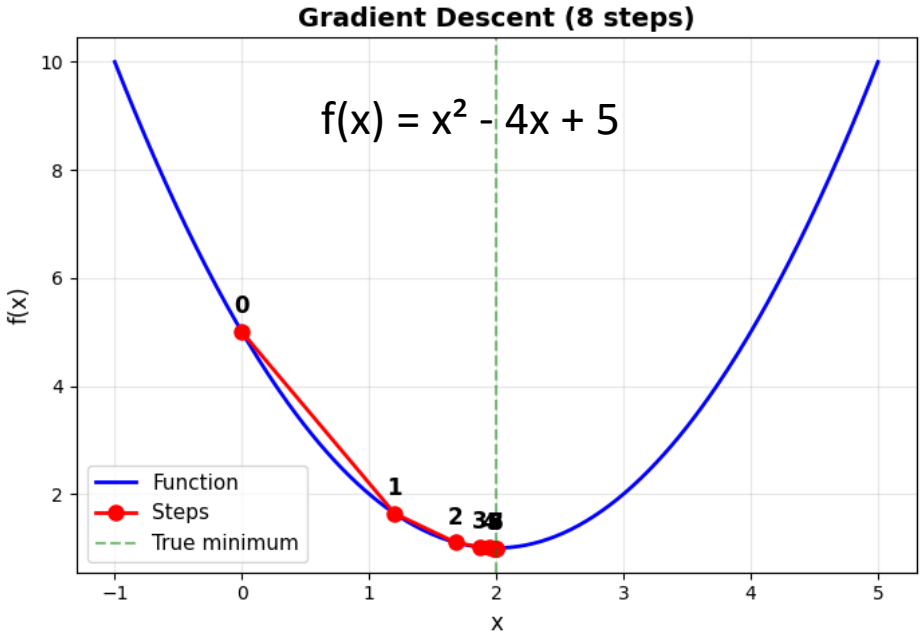


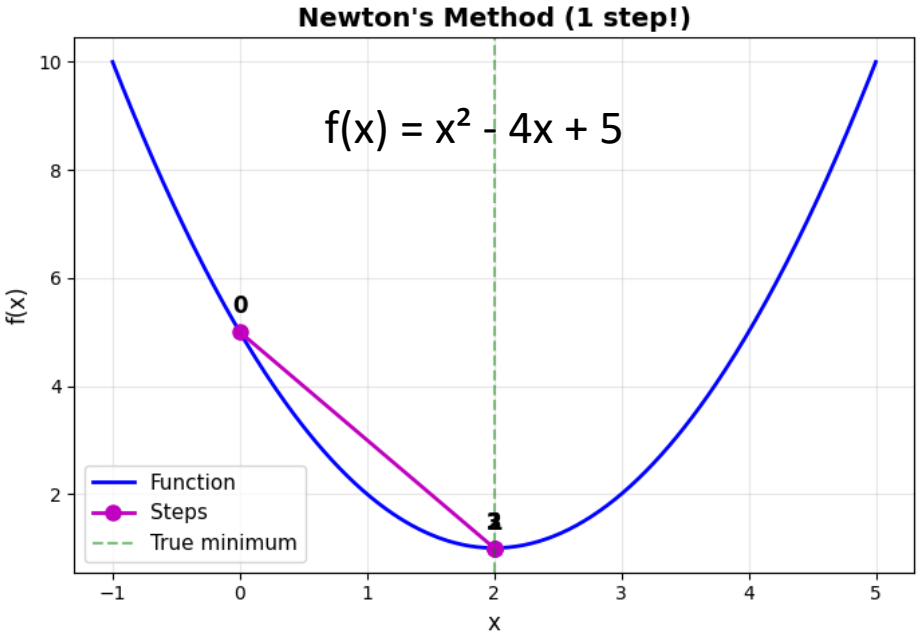
Second-order optimization

Newton's Method (Newton–Raphson method)

$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

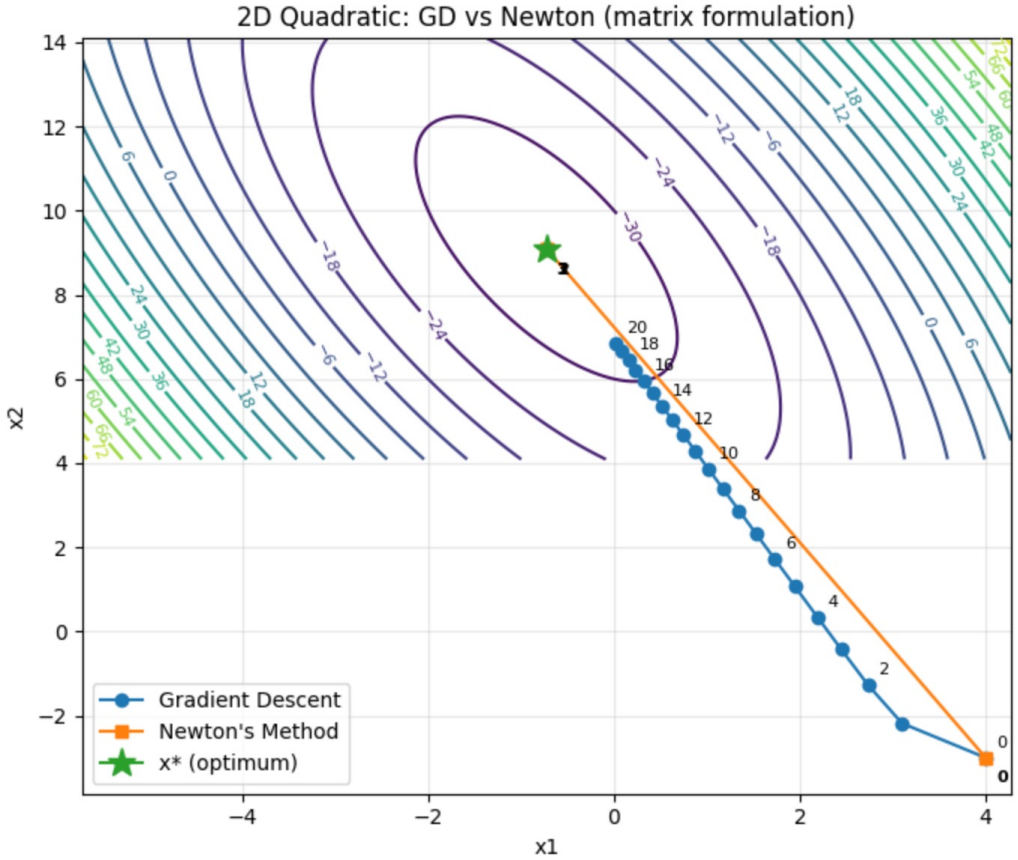
$$\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - f'/f''$$





Second-order optimization

Newton's Method (Newton–Raphson method)



Second-order optimization

Quasi-Newton Methods/ BFGS (Broyden–Fletcher–Goldfarb–Shanno algorithm)

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - B_k^{-1}\nabla f(\boldsymbol{x}_k)$$

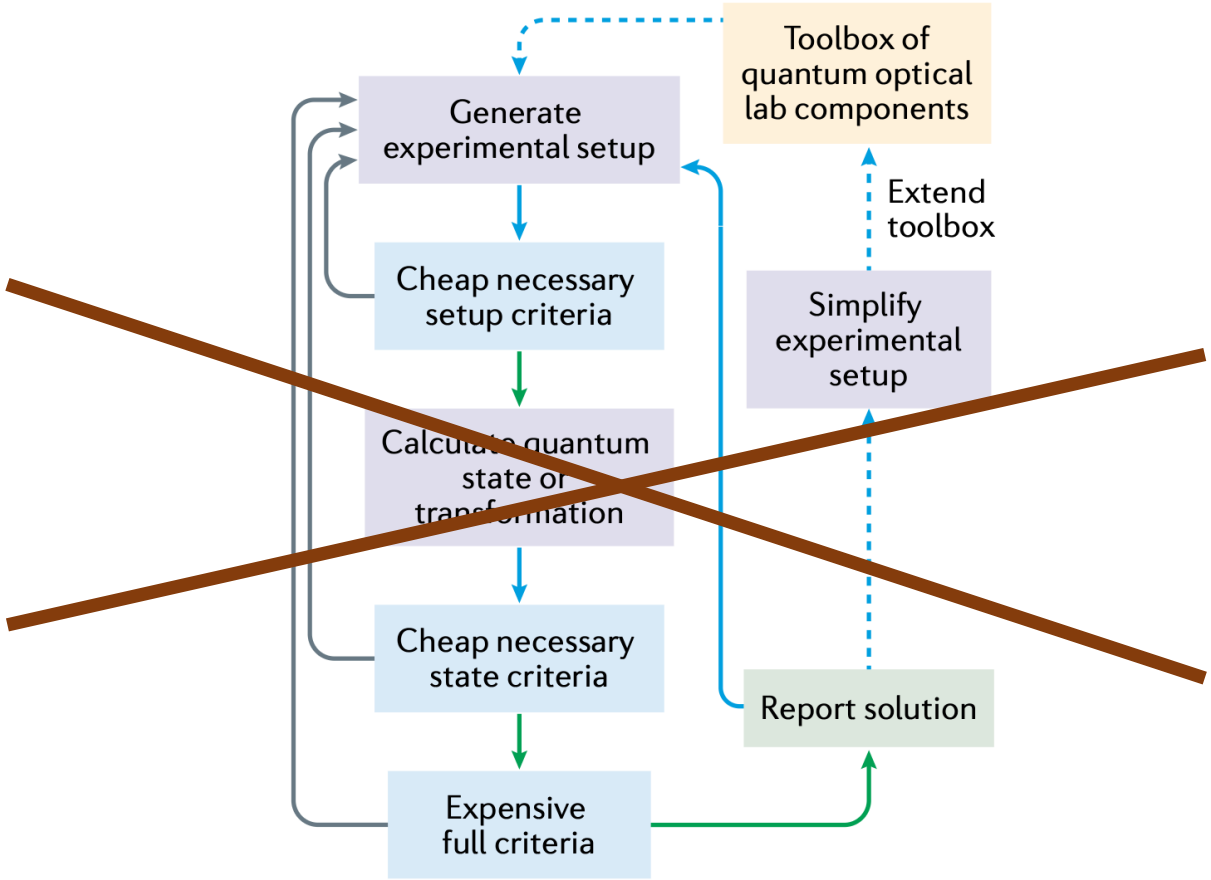
$$B_{k+1}^{-1} = (I - \rho_k s_k y_k^T)B_k^{-1}(I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T$$

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f_{k+1} - \nabla f_k, \quad \rho_k = \frac{1}{y_k^T s_k}$$








Toolbox of quantum optical lab components
Generate experimental setup
Extend toolbox
Cheap necessary setup criteria
Simplify experimental setup
Cheap necessary state criteria
Report solution
Expensive full criteria

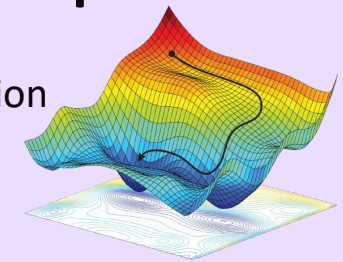
Ansatz-based space + physical simulators + gradient-based optimization

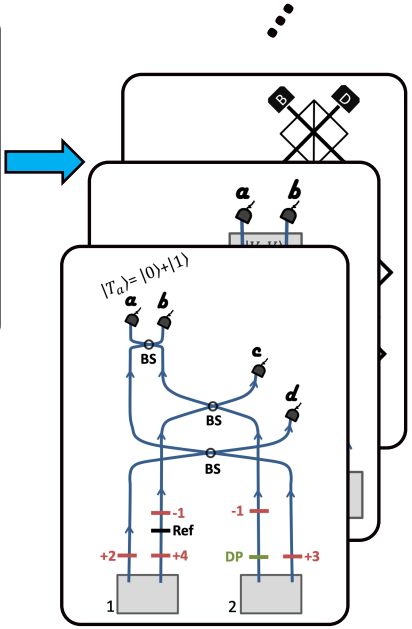
Search space



AI-based Exploration

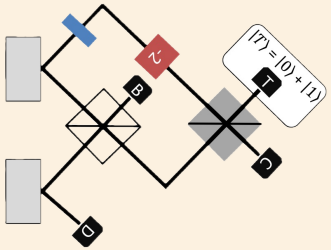
e.g., optimization algorithms





AI-designed setups

Physical Simulator



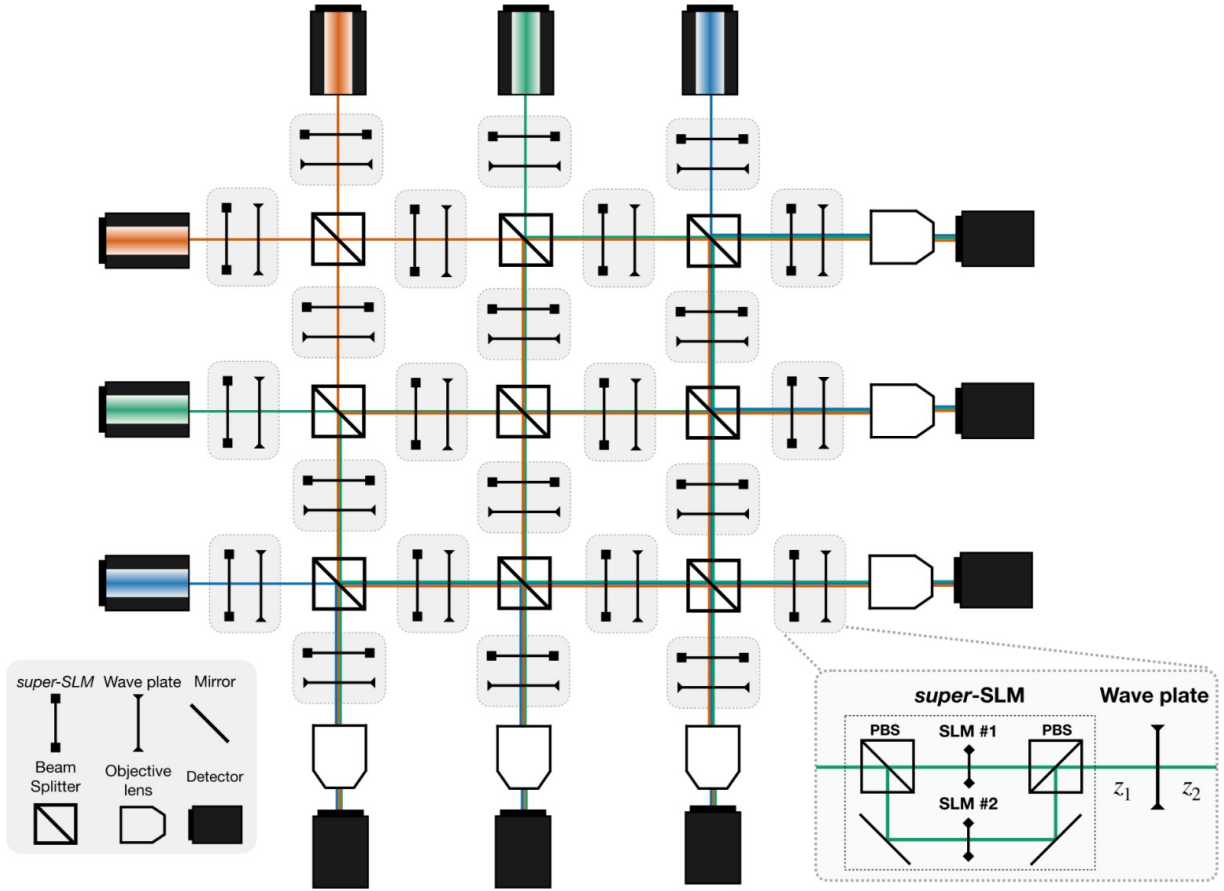
Computed measurement results

Scientific Objective

Computational quality measure

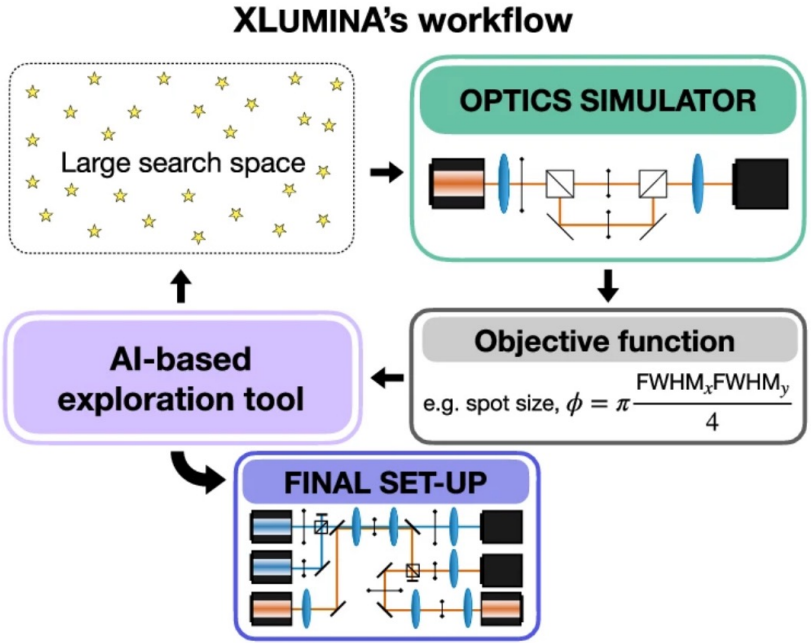
e.g., quantum state's fidelity $|\langle\phi|\Psi\rangle|^2$

Classical microscopy : https://github.com/artificial-scientist-lab/XLuminA



Example: https://github.com/artificial-scientist-lab/XLuminA

Classical microscopy




```python
@jit
def jones_LP(alpha:float):
    """
    Define the Jones matrix of a Linear polarizer.

    Parameters:
        alpha (float): Transmission angle w.r.t. horizontal (in radians).

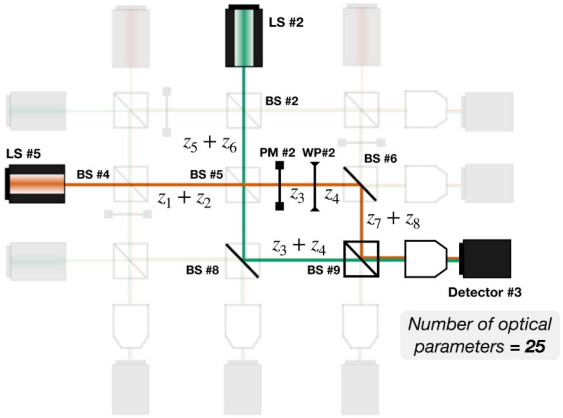
    Returns the Jones matrix (jnp.array).
    """
    return jnp.array([[jnp.cos(alpha) ** 2, jnp.cos(alpha) * jnp.sin(alpha)],
                      [jnp.cos(alpha) * jnp.sin(alpha), jnp.sin(alpha) ** 2]])
```

```python
def clip_adamw(learning_rate, weight_decay) -> optax.GradientTransformation:
    """
    Custom optimizer - adamw: applies several transformations in sequence
    1) Apply ADAM
    2) Apply weight decay
    """
    return optax.adamw(learning_rate=learning_rate, weight_decay=weight_decay)
```

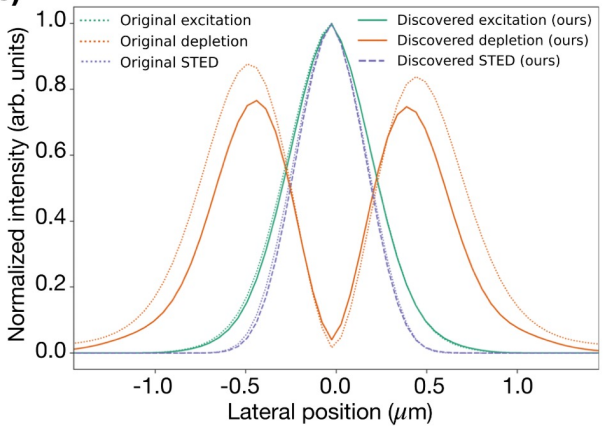

Rodríguez, C., Arlt, S., Möckl, L. *et al.* Automated discovery of experimental designs in super-resolution microscopy with XLuminA. *Nat Commun* **15**, 10658 (2024)
Deb, Diptodip, et al. Chromatix: a differentiable, GPU-accelerated wave-optics library. bioRxiv 2025.04.29.651152. https://doi.org/10.1101/2025.04.29.651152

Classical microscopy :

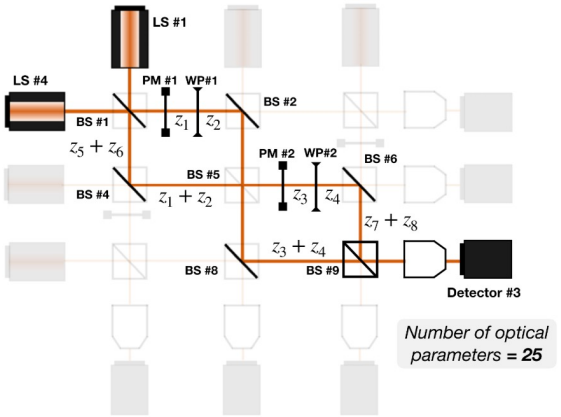
a) Pure topological discovery for Hell and Wichmann (1994)



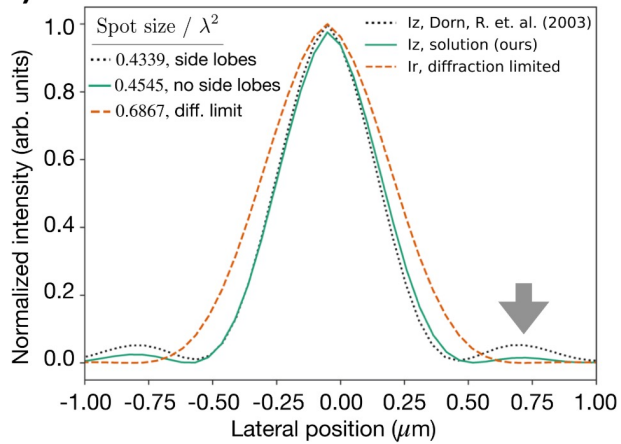
b) Radial intensity profile at focal plane



c) Pure topological discovery for Dorn, Quabis and Leuchs (2003)

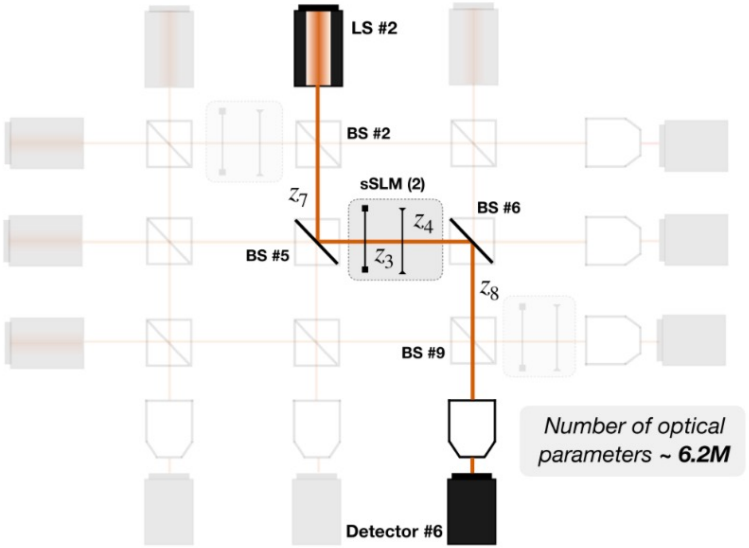


d) Longitudinal intensity profile at focal plane

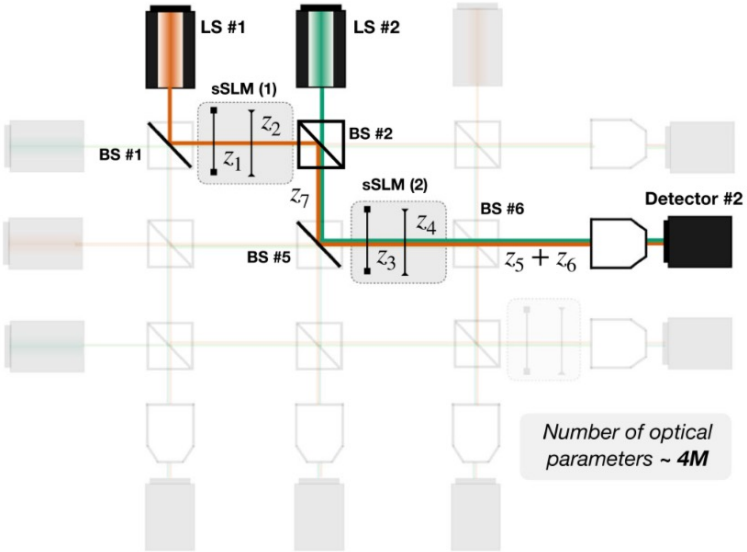


Classical microscopy :

Discovered optical layout for Dorn, Quabis and Leuchs (2003)



Discovered optical layout

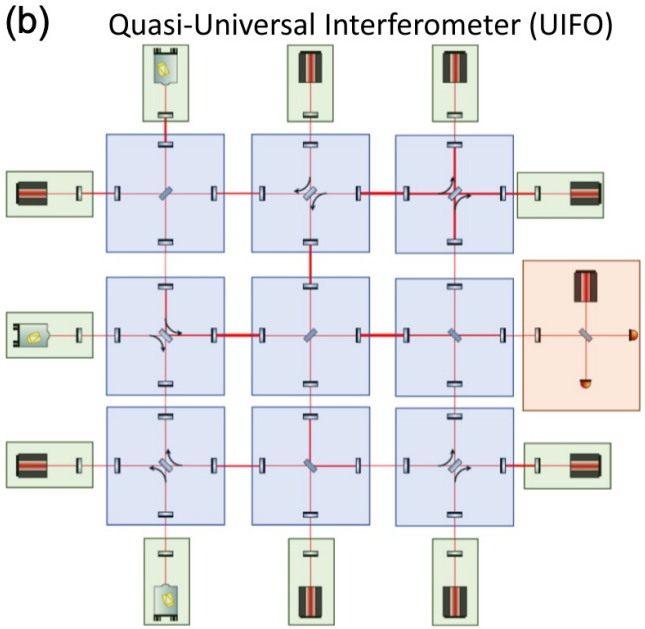


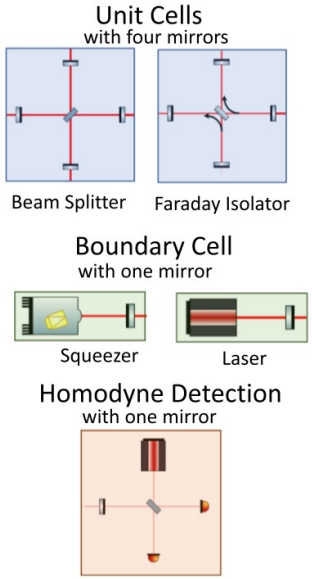
Example: https://github.com/artificial-scientist-lab/GWDetectorZoo

LIGO - gravitational wave detectors

a highly expressive parametrized template for an interferometer

https://finesse.ifosim.org/docs/develop/index.html





Finesse 3

```python
from finesse import Model

model = Model()

# Parse the simple model from above
model.parse(
    """
    # Example of 10 W of laser power incident upon a Fabry-Perot cavity.
    l l1 P=10
    s s1 l1.p1 m1.p1
    m m1 R=0.99 T=0.01
    s s2 m1.p2 m2.p1
    m m2 R=0.99 T=0.01
    pd pd1 m2.p1.i
    modes(even, maxtem=10)
    """
)
```

internal: Matrix system

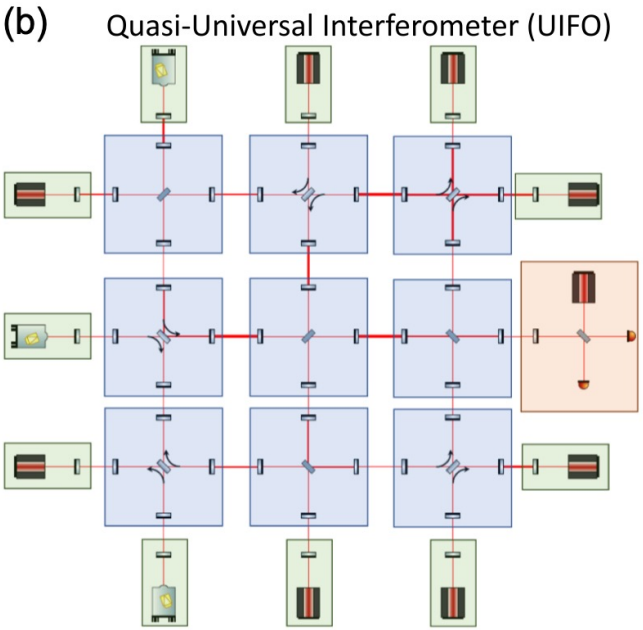
Physical Simulator

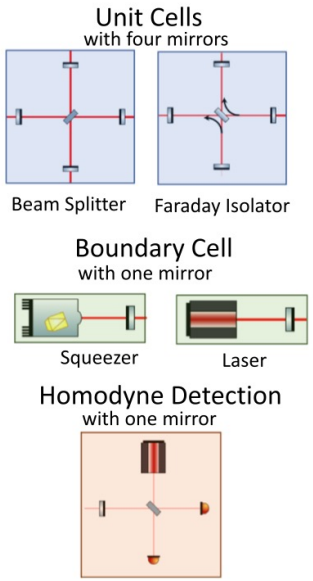
Krenn, Mario, Yehonathan Drori, and Rana X. Adhikari. Digital discovery of interferometric gravitational wave detectors. *PRX*: 021012 (2025).

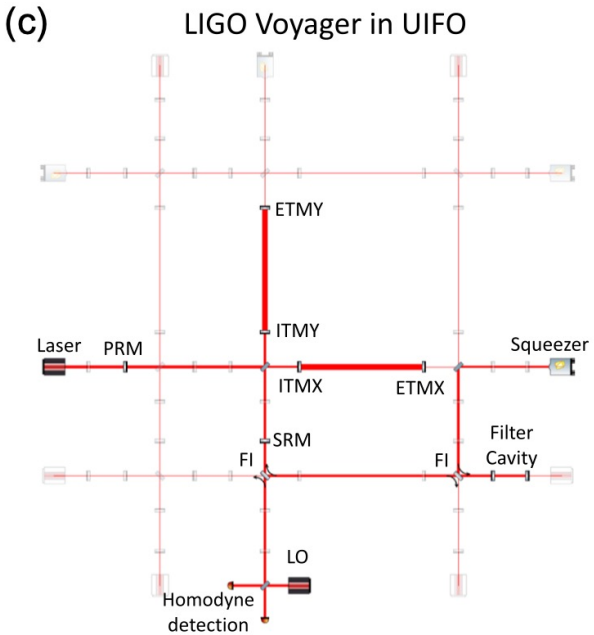
Example: https://github.com/artificial-scientist-lab/GWDetectorZoo

LIGO - gravitational wave detectors

a highly expressive parametrized template for an interferometer







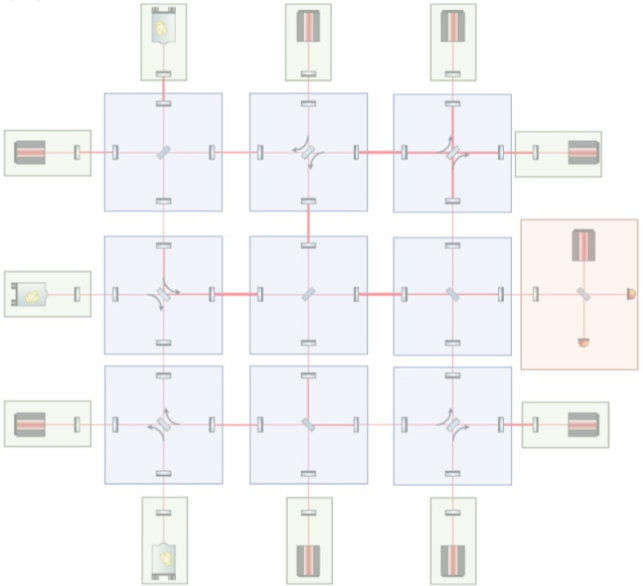
Krenn, Mario, Yehonathan Drori, and Rana X. Adhikari. Digital discovery of interferometric gravitational wave detectors. *PRX*: 021012 (2025).

Example: https://github.com/artificial-scientist-lab/GWDetectorZoo

LIGO - gravitational wave detectors

a highly expressive parametrized template for an interferometer

(b) Quasi-Universal Interferometer (UIFO)



(c) LIGO Voyager in UIFO

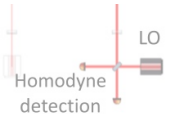


$$\text{Loss}_{\text{total}} = -\text{Loss}_{\text{Strain}} + \alpha\text{Pen}_{\text{damage}} + \beta\text{Pen}_{\text{bleach}}$$

with

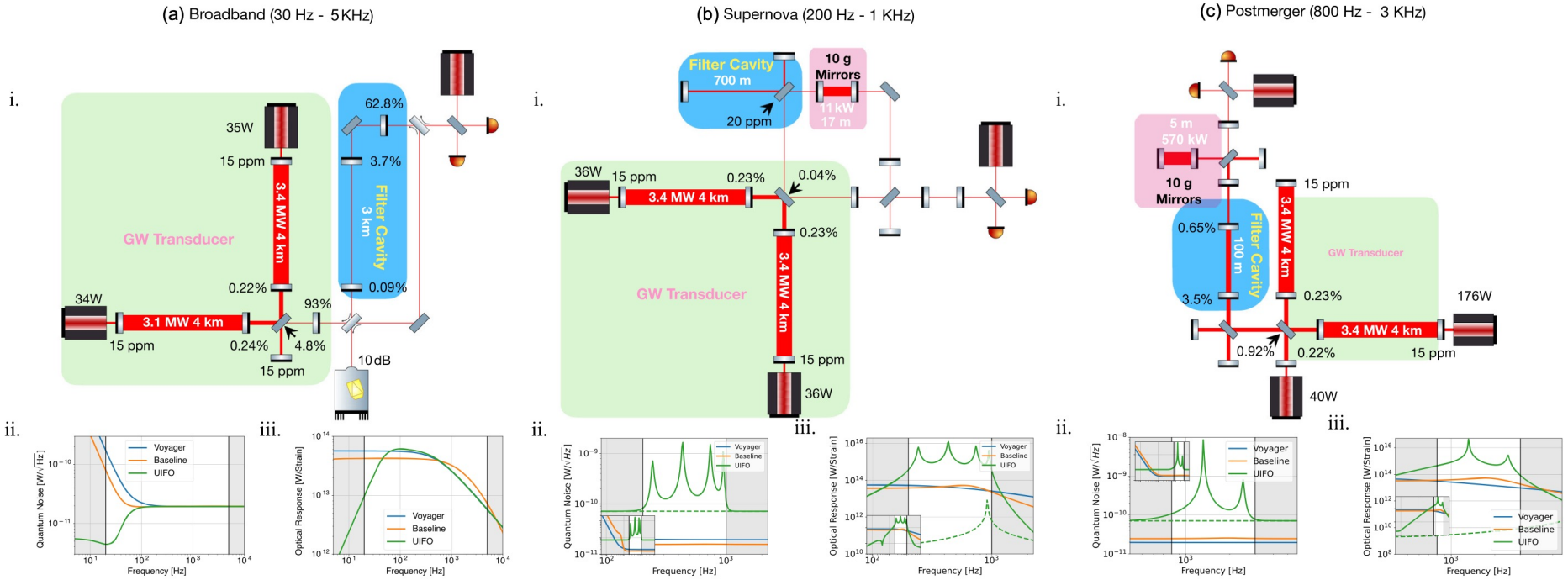
$$\text{Loss}_{\text{Strain}} \approx \int_{f_0}^{f_1} \log\left(S(f)\right) df,$$





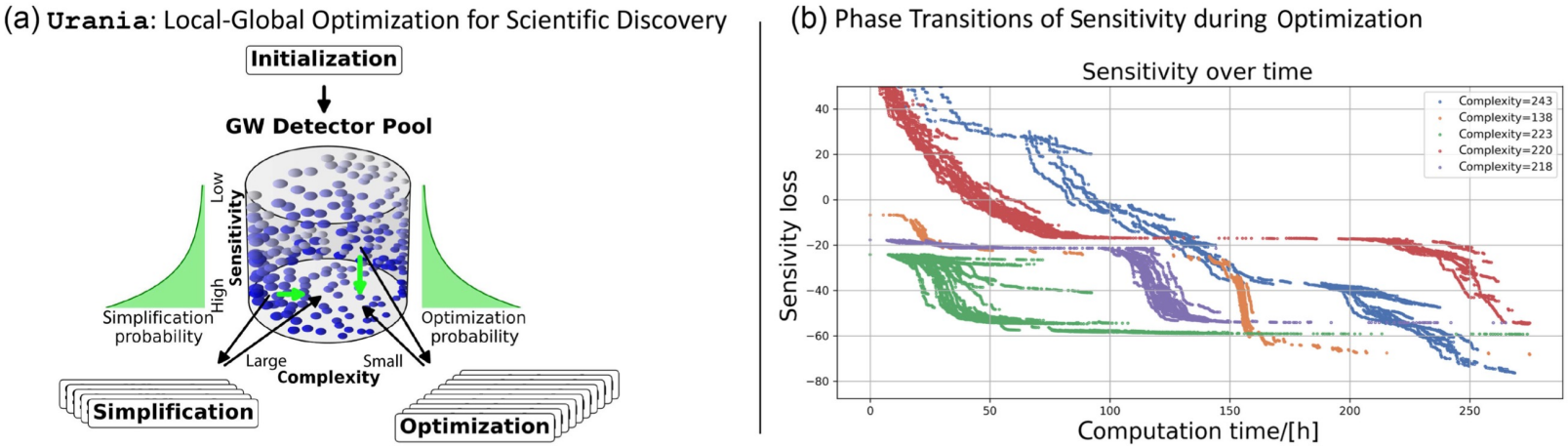
Krenn, Mario, Yehonathan Drori, and Rana X. Adhikari. Digital discovery of interferometric gravitational wave detectors. *PRX*: 021012 (2025).

Example: https://github.com/artificial-scientist-lab/GWDetectorZoo



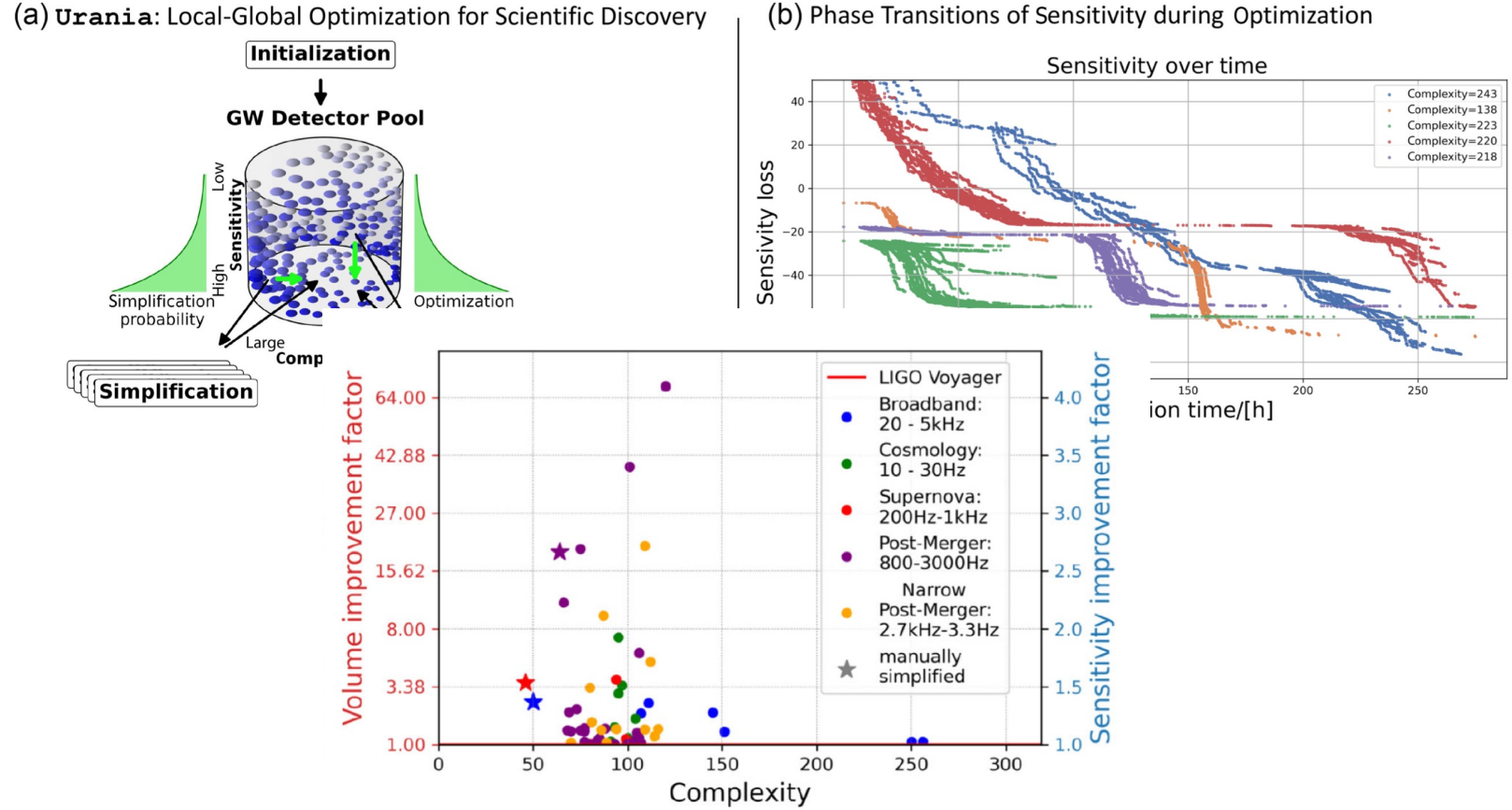
Krenn, Mario, Yehonathan Drori, and Rana X. Adhikari. Digital discovery of interferometric gravitational wave detectors. *PRX*: 021012 (2025).

Example: https://github.com/artificial-scientist-lab/GWDetectorZoo



Krenn, Mario, Yehonathan Drori, and Rana X. Adhikari. Digital discovery of interferometric gravitational wave detectors. *PRX*: 021012 (2025).

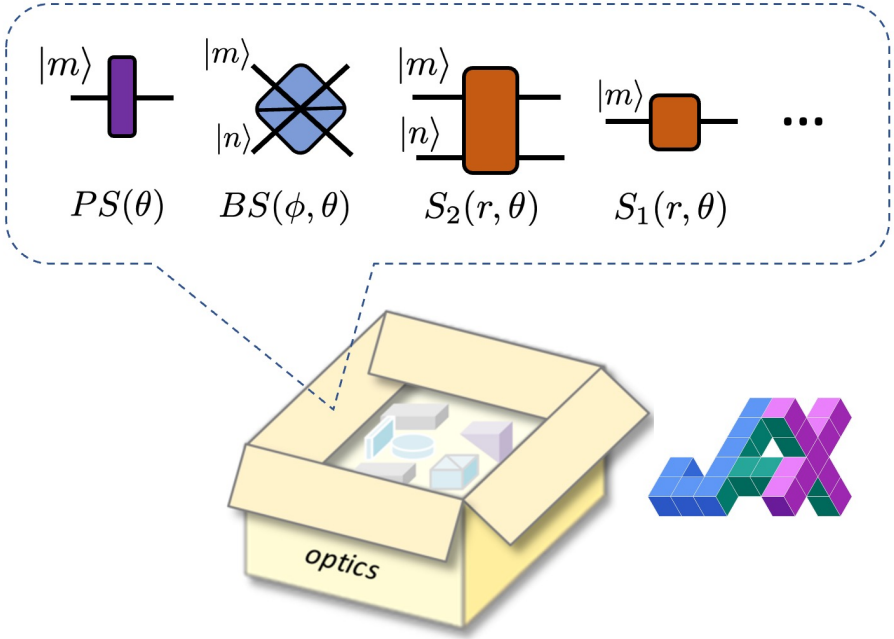
Example: https://github.com/artificial-scientist-lab/GWDetectorZoo



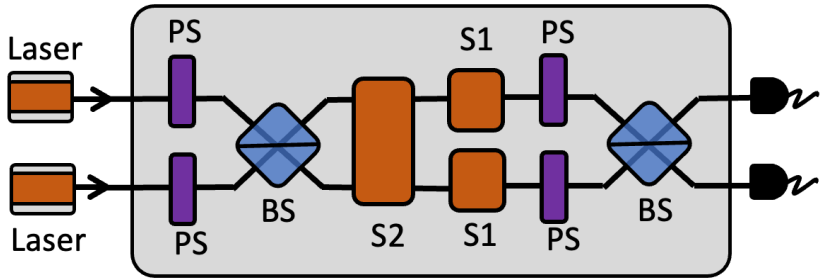
Krenn, Mario, Yehonathan Drori, and Rana X. Adhikari. Digital discovery of interferometric gravitational wave detectors. *PRX*: 021012 (2025).

We can do something similar for Quantum experiments discovery

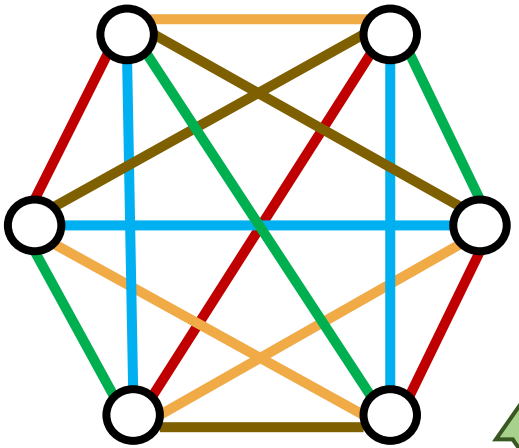
quantum experiments



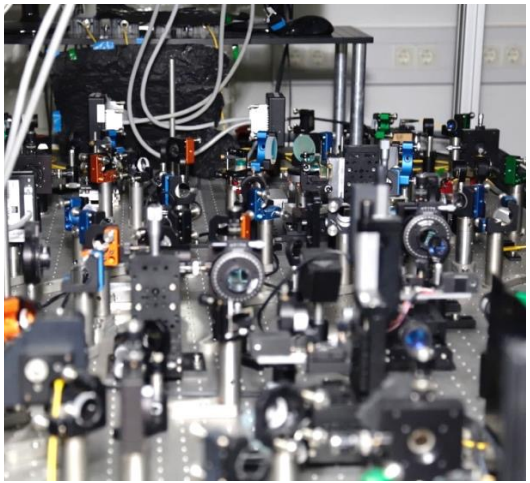
small Ansatz:



New representation of quantum experiments



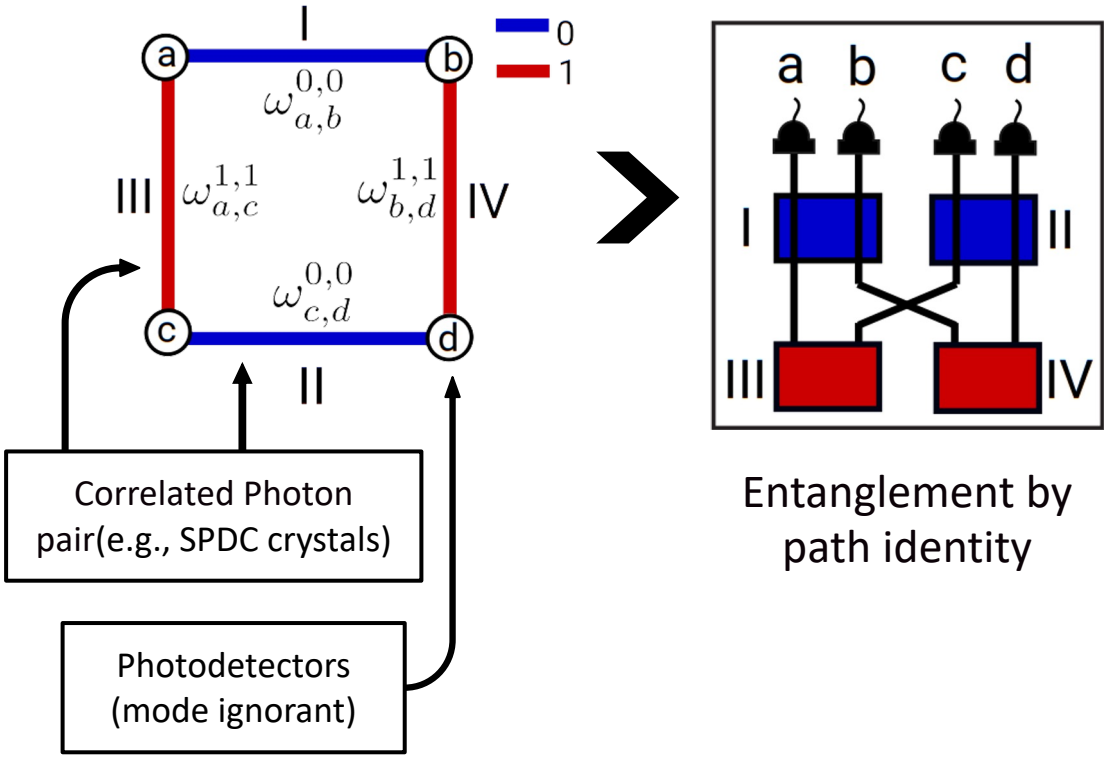
Vertex: Photonic path to detector
Edge: Correlated Photon pair
Edge weight: amplitude
Color: Photonic Mode



M. Krenn, X. Gu, A. Zeilinger, Phys. Rev. Lett. 119, 240403 (2017)
X. Gu, M. Erhard, A. Zeilinger, M. Krenn. PNAS 116 (10), 4147-4155 (2019)

Graph representation

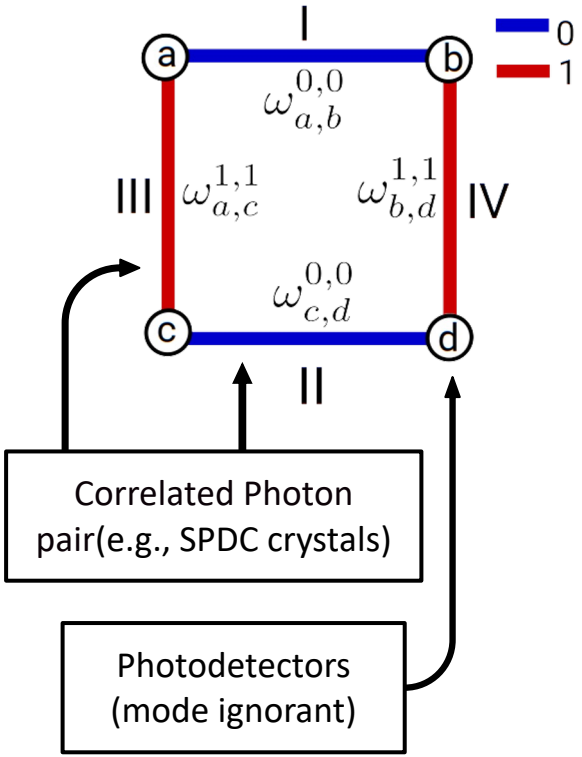
Vertex: Photonic path to detector
Edge: Correlated Photon pair
Edge weight: amplitude
Color: Photonic Mode

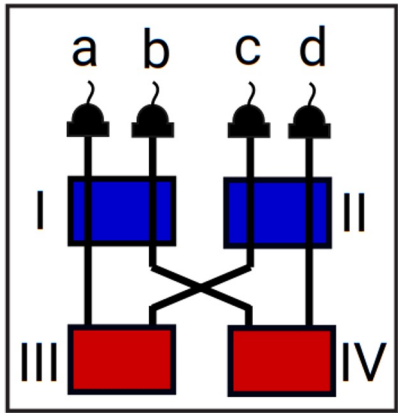


Entanglement by path identity

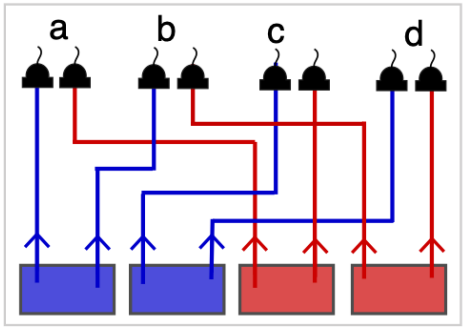
Graph representation

Vertex: Photonic path to detector
Edge: Correlated Photon pair
Edge weight: amplitude
Color: Photonic Mode

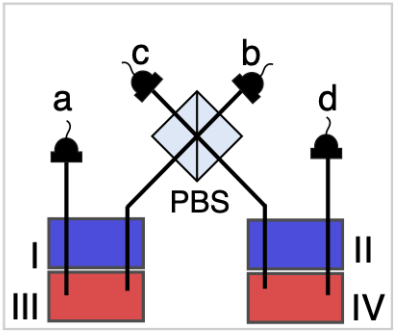




Entanglement by path identity



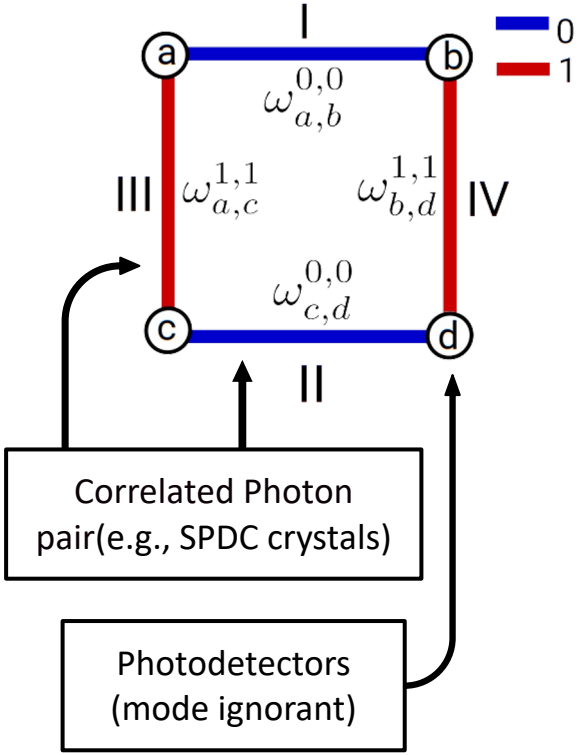
Integrated photonics
i.e., path encoding

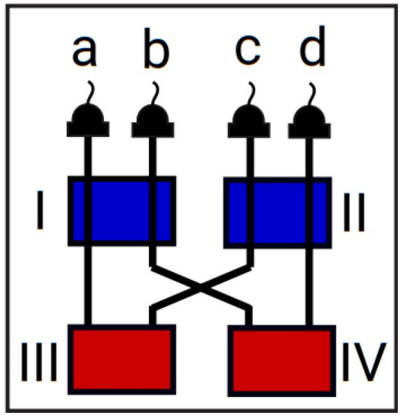


Bulk optics

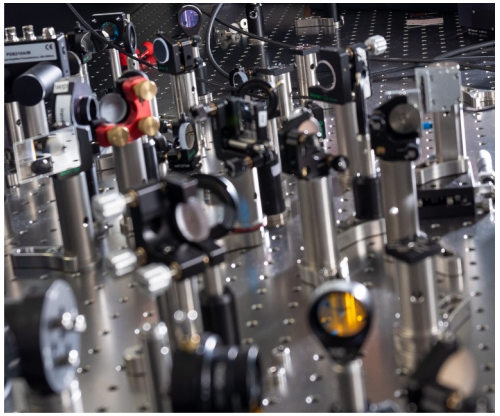
Graph representation

Vertex: Photonic path to detector
Edge: Correlated Photon pair
Edge weight: amplitude
Color: Photonic Mode



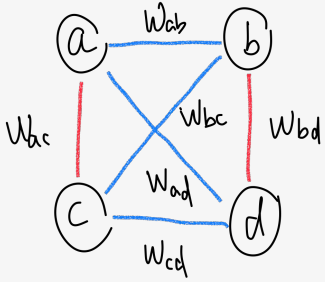


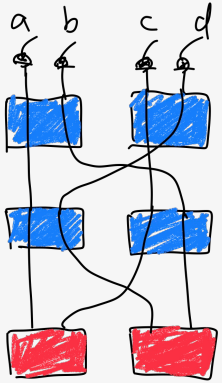
Entanglement by path identity

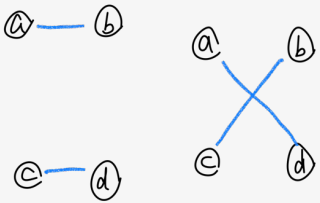


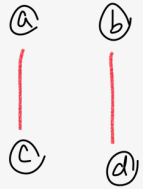
$$|\psi\rangle \approx \omega_{a,b}^{0,0} \cdot \omega_{c,d}^{0,0} |0,0,0,0\rangle + \omega_{a,c}^{1,1} \cdot \omega_{b,d}^{1,1} |1,1,1,1\rangle$$

State generation

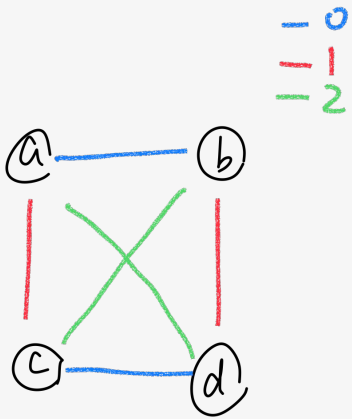


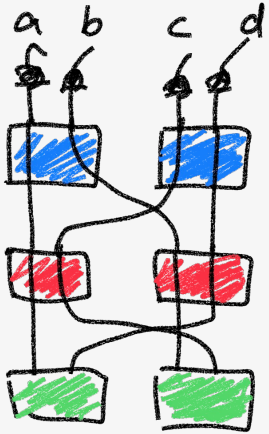






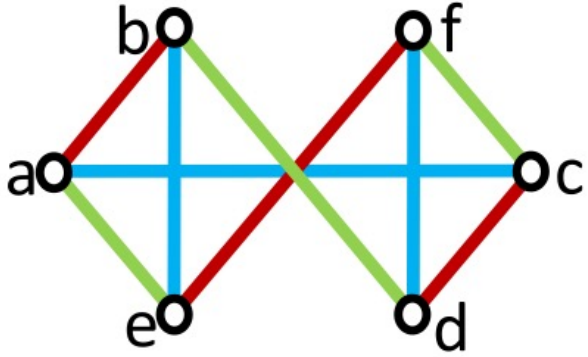
$$(w_{ab} \cdot w_{cd} + w_{bc} \cdot w_{ad})|0000\rangle + w_{ac} w_{bd} |1111\rangle$$

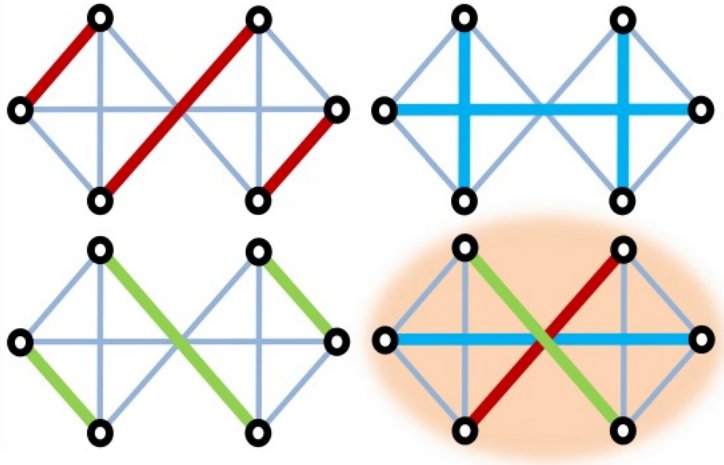




$$w_{ab} \cdot w_{cd} |0000\rangle + w_{ac} \cdot w_{bd} |1111\rangle + w_{ad} \cdot w_{bc} |2222\rangle$$

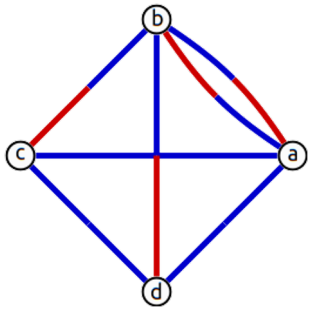
State generation





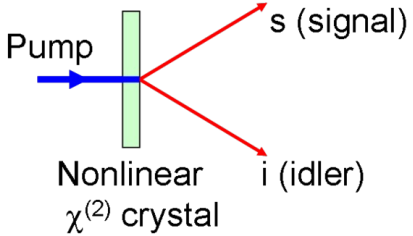
$$|\psi\rangle = \frac{1}{2}(|0,0,0,0,0,0\rangle + |1,1,1,1,1,1\rangle + |2,2,2,2,2,2\rangle + |1,2,1,2,0,0\rangle).$$

$$|W_4\rangle = \Phi(\omega)|vac\rangle$$

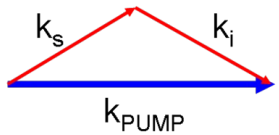


$$\Phi(\boldsymbol{\omega}) = \sum_m \frac{1}{m!} \left(\sum_{e \in E(G)} \omega(e) x^\dagger(e) y^\dagger(e) + \text{h.c.} \right)^m$$

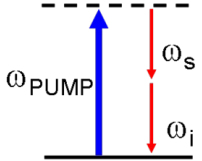
Spontaneous
Parametric
Downconversion



Momentum Conservation



Energy conservation

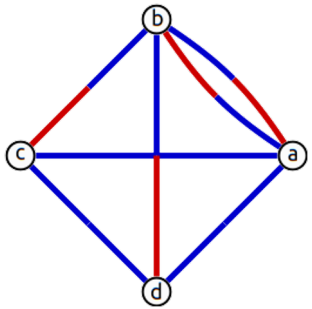


$\varphi_{PUMP} = \varphi_s + \varphi_i$

$$\hat{H} = i\eta(\hat{p}\hat{a}^\dagger\hat{b}^\dagger - p^\dagger\hat{a}\hat{b})$$

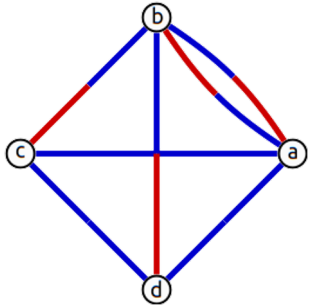
$$U_{\text{SPDC}} = e^{-i\hat{H}t} = \hat{I} + g(\hat{p}\hat{a}^\dagger\hat{b}^\dagger - p^\dagger\hat{a}\hat{b}) + O(g^2)$$
$$\approx \hat{I} + g(\hat{p}\hat{a}^\dagger\hat{b}^\dagger - p^\dagger\hat{a}\hat{b}),$$

$$|W_4\rangle = \Phi(\omega)|vac\rangle$$

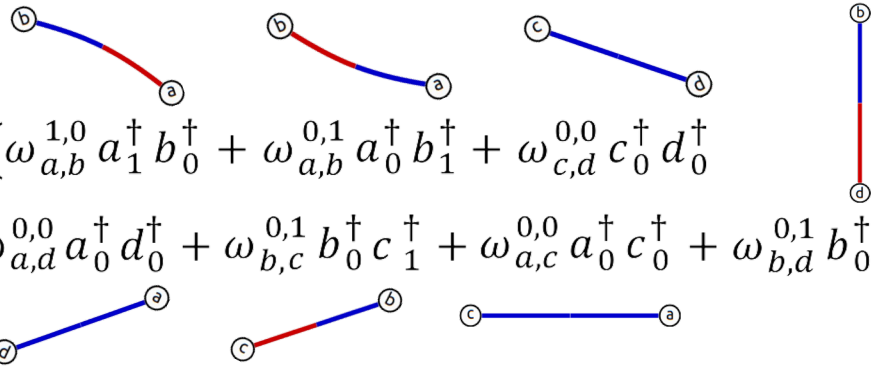


$$\Phi(\omega) \approx \sum_m \frac{1}{m!} \left(\omega_{a,b}^{1,0}\, a_1^\dagger\, b_0^\dagger + \omega_{a,b}^{0,1}\, a_0^\dagger\, b_1^\dagger + \omega_{c,d}^{0,0}\, c_0^\dagger\, d_0^\dagger \right.$$
$$\left. + \omega_{a,d}^{0,0}\, a_0^\dagger\, d_0^\dagger + \omega_{b,c}^{0,1}\, b_0^\dagger\, c_1^\dagger + \omega_{a,c}^{0,0}\, a_0^\dagger\, c_0^\dagger + \omega_{b,d}^{0,1}\, b_0^\dagger\, d_1^\dagger \right)^m$$

$$|W_4\rangle = \Phi(\omega)|vac\rangle$$

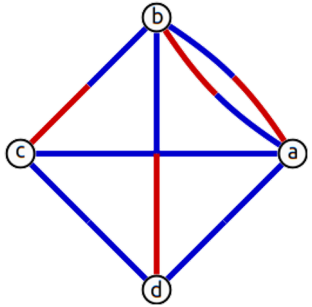


$$\Phi(\omega) \approx \sum_m \frac{1}{m!}\left(\omega_{a,b}^{1,0}\, a_1^\dagger\, b_0^\dagger + \omega_{a,b}^{0,1}\, a_0^\dagger\, b_1^\dagger + \omega_{c,d}^{0,0}\, c_0^\dagger\, d_0^\dagger + \omega_{a,d}^{0,0}\, a_0^\dagger\, d_0^\dagger + \omega_{b,c}^{0,1}\, b_0^\dagger\, c_1^\dagger + \omega_{a,c}^{0,0}\, a_0^\dagger\, c_0^\dagger + \omega_{b,d}^{0,1}\, b_0^\dagger\, d_1^\dagger\right)^m$$

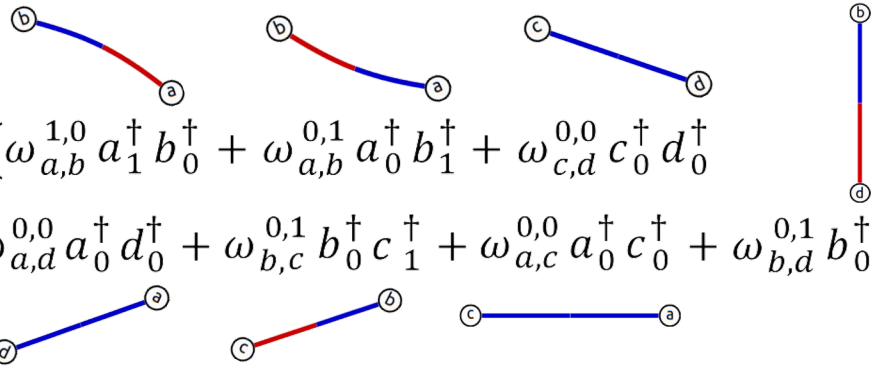


For m = 1

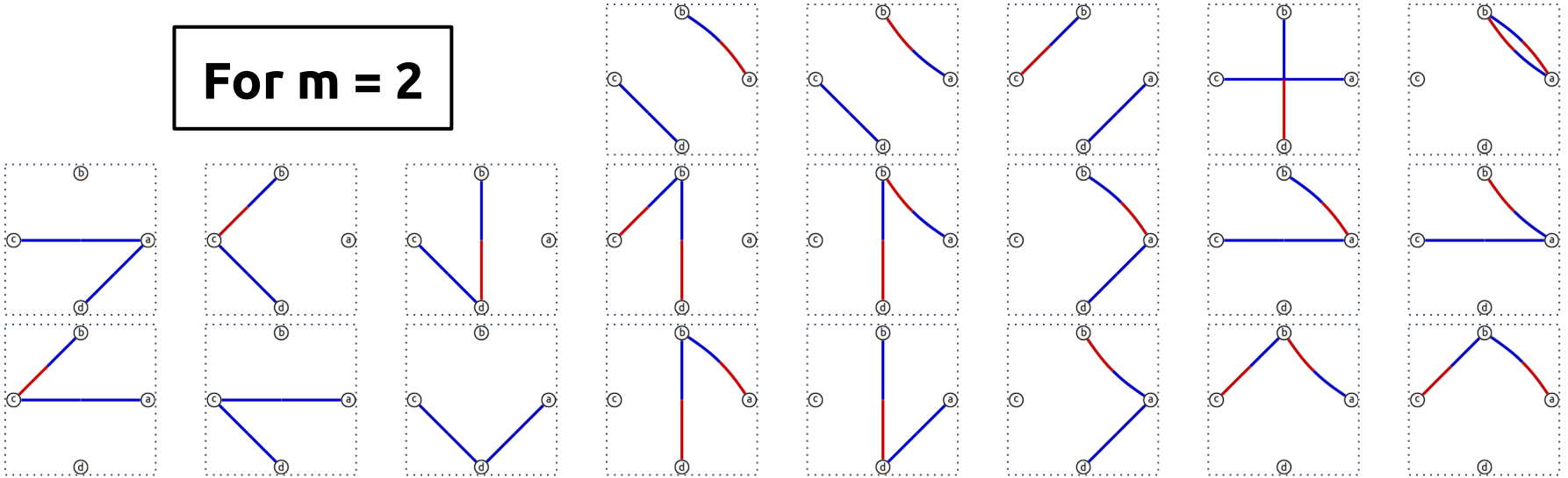
$$|W_4\rangle = \Phi(\omega)|vac\rangle$$



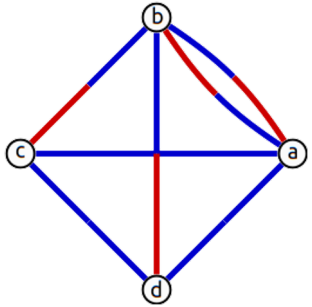
$$\Phi(\omega) \approx \sum_m \frac{1}{m!}\left(\omega_{a,b}^{1,0}\, a_1^\dagger\, b_0^\dagger + \omega_{a,b}^{0,1}\, a_0^\dagger\, b_1^\dagger + \omega_{c,d}^{0,0}\, c_0^\dagger\, d_0^\dagger + \omega_{a,d}^{0,0}\, a_0^\dagger\, d_0^\dagger + \omega_{b,c}^{0,1}\, b_0^\dagger\, c_1^\dagger + \omega_{a,c}^{0,0}\, a_0^\dagger\, c_0^\dagger + \omega_{b,d}^{0,1}\, b_0^\dagger\, d_1^\dagger\right)^m$$



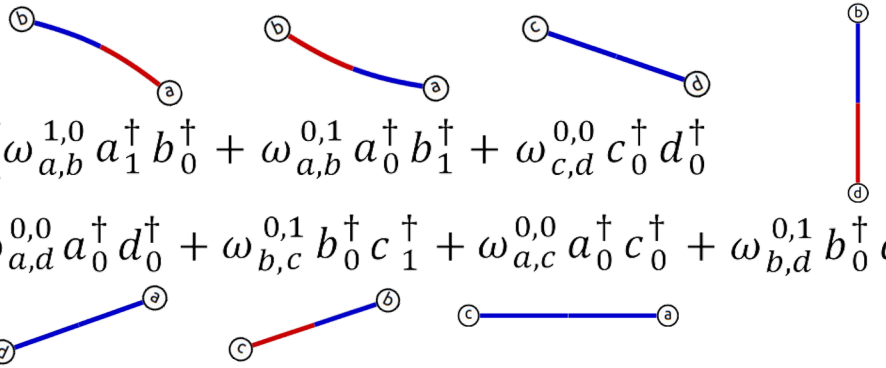
For m = 2



$$|W_4\rangle = \Phi(\omega)|vac\rangle$$



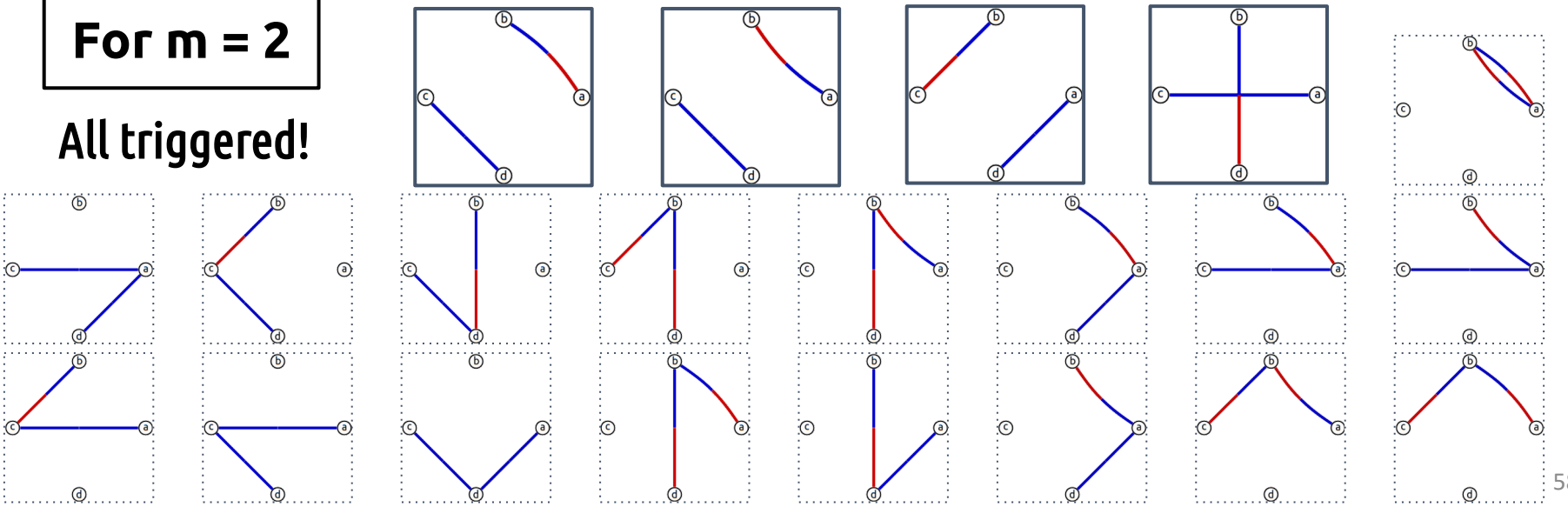
$$\Phi(\omega) \approx \sum_m \frac{1}{m!}\left(\omega_{a,b}^{1,0}\, a_1^\dagger\, b_0^\dagger + \omega_{a,b}^{0,1}\, a_0^\dagger\, b_1^\dagger + \omega_{c,d}^{0,0}\, c_0^\dagger\, d_0^\dagger + \omega_{a,d}^{0,0}\, a_0^\dagger\, d_0^\dagger + \omega_{b,c}^{0,1}\, b_0^\dagger\, c_1^\dagger + \omega_{a,c}^{0,0}\, a_0^\dagger\, c_0^\dagger + \omega_{b,d}^{0,1}\, b_0^\dagger\, d_1^\dagger\right)^m$$



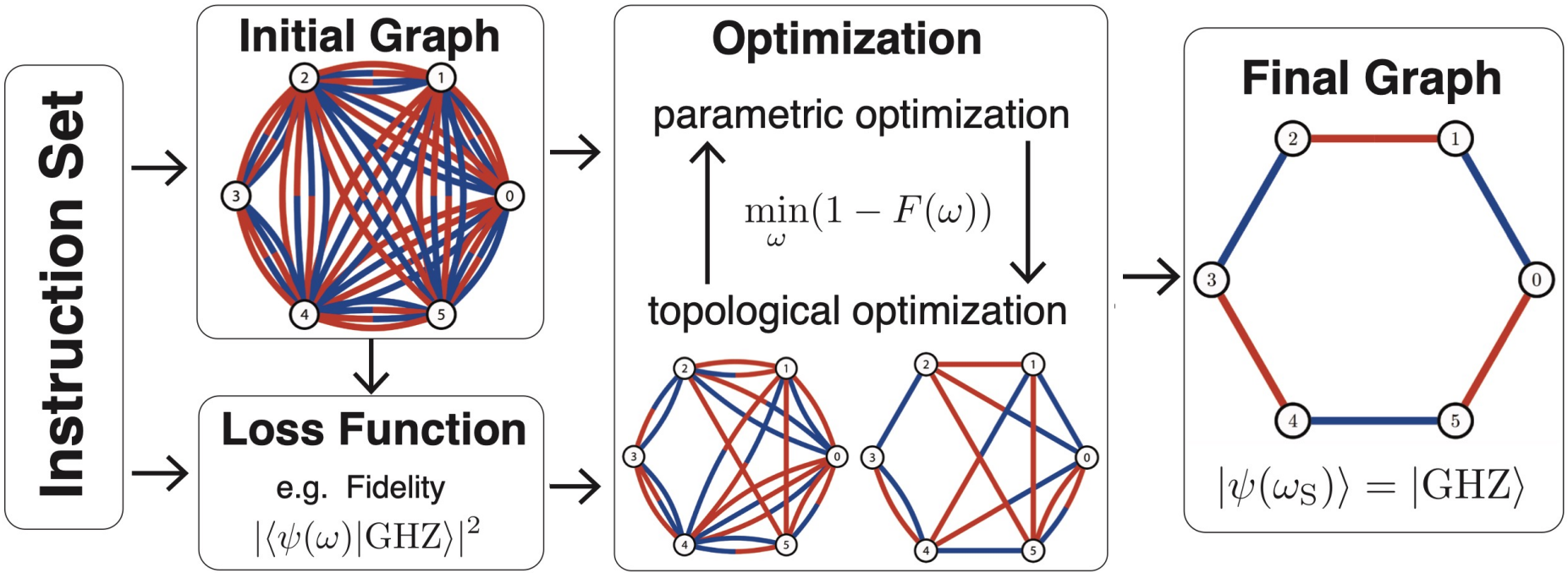
For m = 2

All triggered!

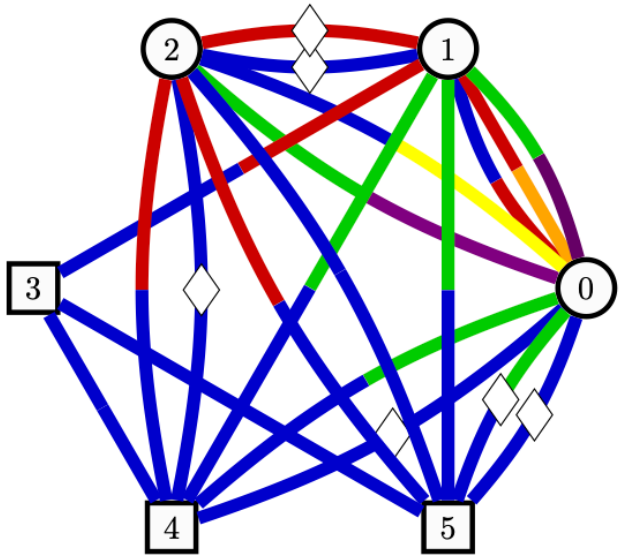
$|1000\rangle$ $|0100\rangle$ $|0010\rangle$ $|0001\rangle$



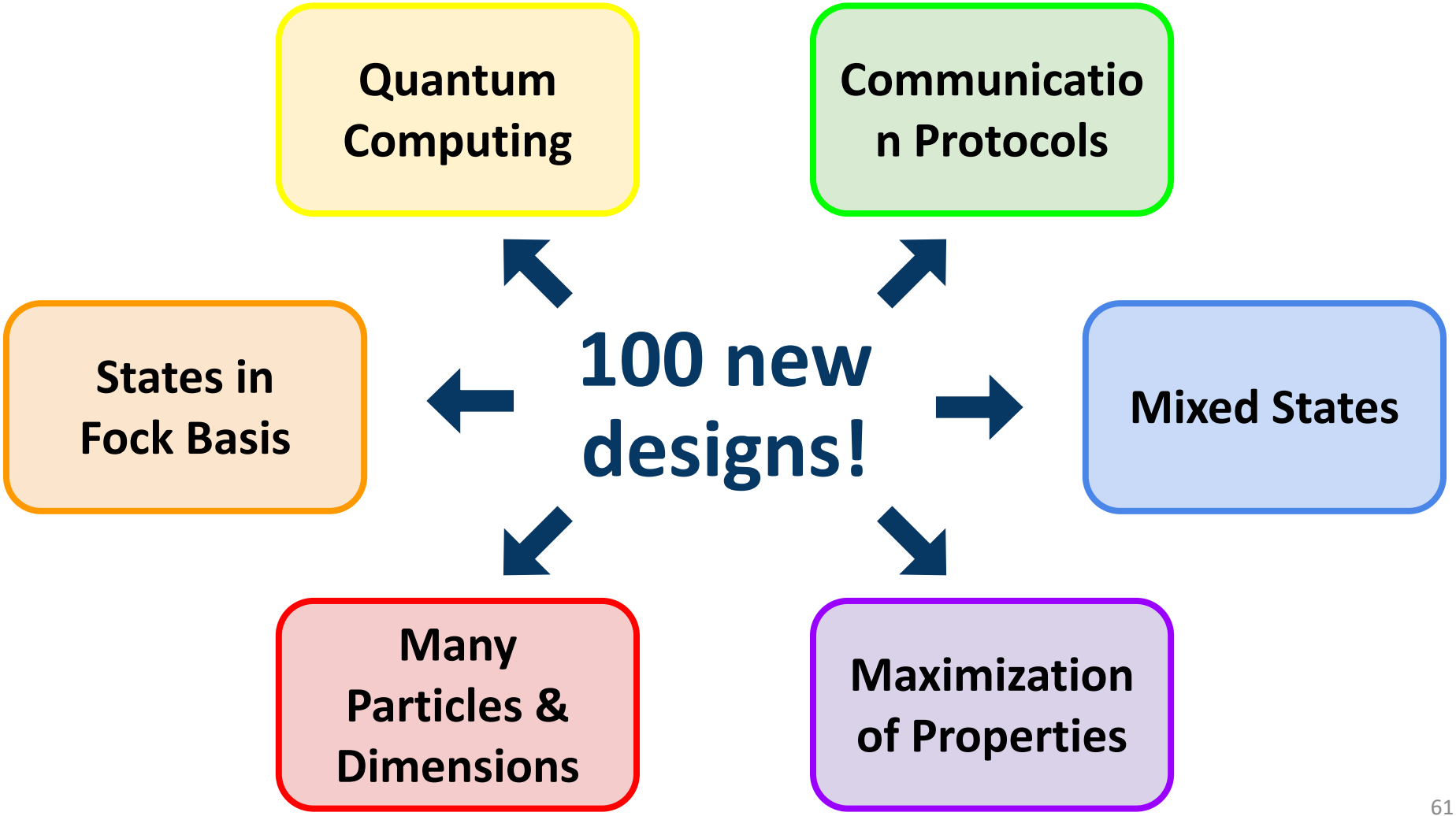
PyTheus Workflow



```
{
    "description": "Creation of state with Schmidt rank vector (7,3,3). Three ancillary particles.",
    "bulk_thr": 0.01,
    "edges_tried": 30,
    "foldername": "srv_733",
    "ftol": 1e-06,
    "loss_func": "cr",
    "num_anc": 3,
    "num_pre": 1,
    "optimizer": "L-BFGS-B",
    "imaginary": false,
    "real": true,
    "safe_hist": true,
    "samples": 10,
    "target_state": [
        "000",
        "101",
        "210",
        "311",
        "422",
        "520",
        "621"
    ],
    "thresholds": [
        0.3,
        0.1
    ],
    "tries_per_edge": 5,
    "unicolor": false
}
```

SRV(7,3,3)


Quantum Computing
Communication Protocols
States in Fock Basis
100 new designs!
Mixed States
Many Particles & Dimensions
Maximization of Properties

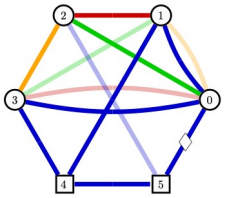
Digital Discovery of 100 diverse Quantum Experiments with PyTheus

github.com/artificial-scientist-lab/PyTheus

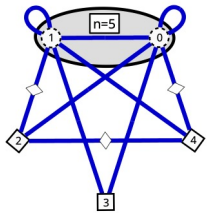
pip install pytheusQ

Carlos Ruiz-Gonzalez[§1], Sören Arlt[§1], Jan Petermann[1], Sharareh Sayyad[1], Tareq Jaouni[2], Ebrahim Karimi[1,2], Nora Tischler[3], **Xuemei Gu** [1], and Mario Krenn[1]

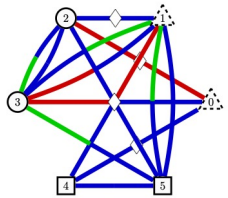
Quantum 7, 1204 (2023)



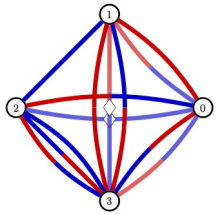
(a) 4-dimensional 4-photon GHZ state (asymptotic and exact solutions).



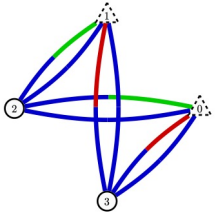
(b) 2-mode 5-photon N00N state $|50\rangle + |05\rangle$ (symmetric graph with an inscribed pentagram).



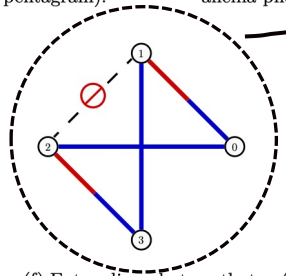
(c) Post-selected CNOT quantum gate acting on a qutrit target with two ancilla photons



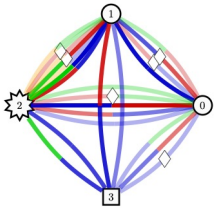
(d) A 4-qubit entangled state which requires weights with the golden ratio (or complex-valued) to be generated.



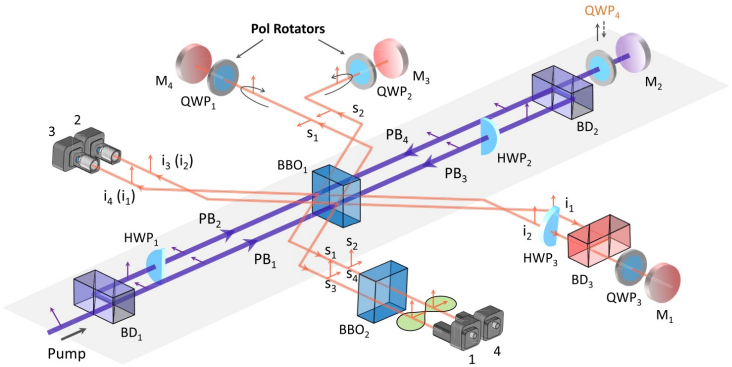
(e) Quantum measurement for a quantum communication task with quantum advantage (Mean King's Problem).



(f) Entangling photons that never interacted (related to entanglement swapping) without initial Bell states.



(g) Mixed state with bound entanglement that violates Bell inequality (counterexample to the Peres conjecture from 1999, solved 2014).



Wang, K., et al. Entangling Independent Particles by Path Identity, Phys. Rev. Lett. **133**, 233601 (2024)

1) Finding high-dimensional 3-partite Entanglement:

States such as: $|\psi\rangle = \frac{1}{2}(|0,0,0\rangle + |1,0,1\rangle + |2,1,0\rangle + |3,1,1\rangle)$

algorithm	**Type**	**Time**
PRL **116**, 090405 (2016)	Topological search + learning	150 hours
THESEUS/PyTheus	Graph-Optimizer	**2 sec**

2) High-dimensional CNOT gate

algorithm	**Type**	**Time**
PRL **125**, 050501 (2020)	Topological search	150.000 CPU-h
THESEUS/PyTheus	Graph-Optimizer	**1 sec**

Homework:

1. Calculate the SRV(ra, rb, rc) for the following quantum state

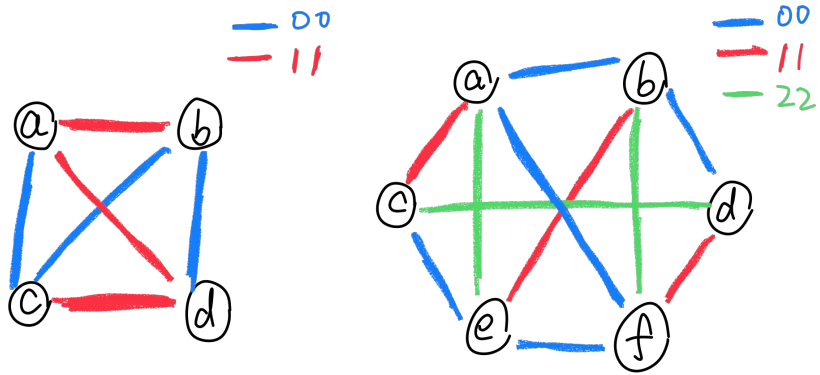
SRV(?,?,?) $|\psi\rangle_{abc} = \frac{1}{2}(|0,0,0\rangle + |1,0,1\rangle + |2,1,0\rangle + |3,1,1\rangle)$

SRV(?,?,?) $|\psi\rangle_{abc} = \frac{1}{\sqrt{5}}(|0,0,0\rangle + |1,1,1\rangle + |2,2,2\rangle + |3,3,3\rangle + |4,4,3\rangle)$

Homework:

2. Translate the following graphs into experimental setups based on **entanglement by path identity**, and compute the corresponding quantum states determined by the perfect matchings of each graph (**assuming all edge weights are 1**).

Try to normalize the states at the end and check whether each state is separable or entangled.



Homework:

3. Derive the update rules for several optimization algorithms: Gradient Descent, Momentum, Nesterov Accelerated Gradient, AdaGrad, RMSProp, and Adam. Chose 2-3 methods you like.

For each method you chose, show how the parameter updates evolve **over three steps** when minimizing a simple one-dimensional function, such as:

$$f(x) = (x - 3)^2$$

Homework:

4. Implement and visualize optimization of two simple functions in SciPy, PyTorch, JAX using:

$$f(x) = x^3 + 6x + 2$$

$$f(x, y) = (x - 1)^2 + 10\,(y + 2)^2$$

- Momentum
- Adam
- BFGS

Plot the optimization trajectories f vs. steps for each method and framework. You may directly use the built-in optimizers or implement the update rules yourself. Feel free to experiment with other methods (e.g., RMSProp, Nesterov, L-BFGS).

You can use colab if you don't want to install any packages in your local computers.

Code from the lecture:
https://colab.research.google.com/drive/1f-2LzAl2KGIWNuvkU00UZE-nky745v9H?usp=sharing